CHAPTER 1

INTRODUCTION AND BACKGROUND OF INDUSTRY

1.1 Introduction:

Rural areas are rich in traditional skills and handmade products, but artisans often face challenges in reaching a wider market due to lack of awareness, poor infrastructure, and dependency on middlemen. As a result, they do not receive fair value for their products. With the growth of digital technology, e-commerce platforms have become an effective way to connect sellers and buyers, but rural artisans are still not fully integrated into these systems.

The “Rural E-Commerce & Logistics System” is a web application designed to overcome these challenges by providing a platform where artisans can directly sell their products to customers. The system creates a digital environment that supports buying, selling, and managing rural products efficiently. It reduces the gap between rural producers and urban consumers by eliminating intermediaries and ensuring better pricing and transparency.

The system consists of three main modules: Admin, User, and Artisan. Each module plays an important role in managing activities such as product listing, order placement, payment, and delivery. In addition, the system includes a logistics component that helps in tracking orders and ensuring timely delivery of products from rural areas to customers.

The main aim of this system is to promote digital inclusion in rural areas, improve employment opportunities for artisans, and support the growth of rural businesses. It also simplifies the overall process by providing a user-friendly interface, making it easy for all users to interact with the system. Through this platform, rural artisans can expand their reach, increase their income, and become part of the digital economy.

1.2 Background of the Industry:

With the rapid growth of digital technology and e-commerce platforms, the buying and selling process has become easier and more efficient. However, rural artisans still face many challenges such as lack of market access, poor infrastructure, limited digital knowledge, and dependency on middlemen. Due to these problems, they are unable to reach a wider audience and do not get fair value for their products. The “Rural E-Commerce & Logistics System” is

developed as a purposeful project activity designed to solve real-world problems faced by rural communities. This project helps students to apply their technical knowledge and skills in designing and developing a complete web-based system. It integrates various concepts such as database management, user interface design, and system development to achieve higher-level learning outcomes.

Capstone Project

A capstone project can be understood as the final and most important academic activity that represents the culmination of knowledge and skills acquired throughout the entire program. Just like a “crown” placed on top of a structure symbolizes completion, a capstone project reflects the overall learning and development of a student. It acts as a bridge between theoretical knowledge and real-world application. The “Rural E-Commerce And Logistics System” is developed as a capstone project to address real-world challenges faced by rural artisans by integrating concepts of web development, database management, and system design. It provides an opportunity for students to apply their knowledge in developing a complete system involving Admin, User, and Artisan modules. Capstone projects emphasize experiential learning, problem-solving, and innovation, helping students to gain practical exposure and industry-relevant skills. Thus, it serves as a transition phase that prepares students for professional careers by enhancing their ability to work on real-world problems and making them industry-ready graduates.

CHAPTER 2

LITERATURE SURVEY AND SPECIFICATION

2.1 Study of existing system / Review of research paper:

To build an effective Rural E-Commerce And Logistics System, it is important to understand the current methods used by rural artisans to sell their products and to review existing systems in the e-commerce and logistics domain. This helps in identifying challenges, limitations, and areas where improvements can be made.

Our study of the existing system shows that most rural artisans depend on traditional methods such as local markets, middlemen, and small-scale exhibitions to sell their products. These methods often result in limited reach, low profit margins, and lack of proper pricing. In many cases, artisans do not have access to digital platforms or lack awareness about online selling. Logistics is also a major challenge, as transportation facilities in rural areas are not well-developed, leading to delays in product delivery.

Existing e-commerce platforms mainly focus on urban sellers and do not fully support the needs of rural artisans. There is also a lack of integration between product selling and logistics tracking, making the process inefficient and difficult to manage. Product information, orders, and delivery details are often not maintained properly, which creates confusion and reduces transparency.

We've taken a closer look at how colleges currently manage? Here’s what we've found:

Project Co-Ordinator:

Admin:

- Admin currently manages artisan and user records manually or using basic systems.
- Admin needs to track product listings and order details separately, which is time-consuming.
- Admin monitors delivery and logistics activities without a centralized system.
- Admin handles reports and system activities manually, leading to inefficiency.

Artisan:

Assigned to sell products

- Artisan manages product details manually or through limited tools.
- Artisan faces difficulty in reaching customers directly.
- Artisan tracks orders and sales manually without proper system support.
- Artisan communicates with admin or customers manually for order updates.
- Artisan maintains records of products and earnings manually.

User (Customer):

- User browses products through different platforms without a unified system.
- User places orders but faces difficulty in tracking delivery status properly.
- User maintains purchase records manually or depends on external platforms.
- User does not get real-time updates about order and delivery.
- User follows manual communication with sellers for queries and updates.

Study of exiting E-commerce & Logistics Application

Table 1 Study of existing E-commerce & Logistics Application

Sr No.	Application	Description
1.	Amazon	This system provides functionalities such as product listing, online shopping, payment system, order tracking, customer reviews, and fast delivery services, but it mainly focuses on large-scale sellers.
2.	Meesho	This system focuses on reselling and small businesses, offering product sharing, order management, and delivery support, but lacks direct artisan empowerment and control.

2.2 Limitations of existing system / Problems discussed in research papers:

i. Dependence on traditional and manual selling methods.

ii. Limited market access for rural artisans.

iii. Limited communication between artisans and customers.

2.3 Problem Identification / Need of a system:

i. Artisans face difficulty in selling products due to lack of digital platforms.

ii. Limited reach to customers and dependency on middlemen reduces profit.

iii. Difficulty in managing product details and inventory manually.

iv. Lack of proper system to track orders and delivery status.

v. Logistics management is inefficient and causes delays in delivery.

2.4 Problem definition:

The proposed system is a web application to streamline the process of buying and selling rural products by connecting artisans directly with customers and integrating logistics support for efficient order management and delivery.

2.5 Specification:

- **Hardware Specification:**
  - Operating System: Windows 10 (64 bit) 0GB
  - Processer: Intel(R) Core(TM) i3 CPU @ 3.30 GHz
  - Ram: 4.00GB
- **Software specification:**
  - Frontend: React.js
  - Backend: Node.js
  - Database: MongoDB
  - Tools: Visual Studio Code
  - Browser: Google Chrome

CHAPTER 3

SCOPE OF PROJECT

The project scope entails developing a comprehensive Rural E-Commerce & Logistics System catering to three distinct user roles: Admin, Artisan, and User (Customer). Admin functionalities include managing users and artisans, monitoring products, handling orders, and overseeing logistics activities. Artisans are empowered to register, manage their profiles, upload products, handle inventory, and process orders. Users (customers) can browse products, place orders, make payments, and track deliveries. The system integrates e-commerce with logistics to ensure smooth product distribution from rural areas to customers. Non-functional requirements include secure user authentication, real-time order tracking, role-based access control, user-friendly interface, and efficient data management. The system aims to promote rural business growth, ensure transparency, and provide efficient communication among all stakeholders.

User roles: Admin, Artisan, and User (Customer):

- Managing users and artisans
- Product listing and management
- Inventory management
- Payment processing
- Monitoring system activities
- Accessing product and order details
- Real-time order status updates
- Role-based access control
- User-friendly interface
- Efficient data management
- Improving rural market access
- Ensuring transparency in transactions
- Streamlining e-commerce and logistics processes
- Enhancing communication between artisans and customers

CHAPTER 4

METHODOLOGY

4.1 User Requirement:

Functional Requirement:

There are three user roles for the system: Admin, Artisan, and User (Customer).

i. Requirement of Admin:

- Admin shall be able to view and manage all users and artisans.
- Admin can approve or reject artisan registrations.
- Admin shall be able to manage product listings and monitor product details.
- Admin shall be able to view and manage all orders placed by users.
- Admin can track order status and manage delivery processes.
- Admin shall be able to assign and monitor logistics activities.
- Admin shall be able to view reports of sales and system activities.
- Admin shall be able to maintain records of products, users, and transactions.

ii. Requirements of Artisan:

- Artisan shall be able to register and login with unique credentials.
- Artisan shall be able to manage their profile details.
- Artisan shall be able to add, update, and delete product details.
- Artisan shall be able to manage product inventory and stock.
- Artisan shall be able to view orders received from customers.
- Artisan shall be able to accept or reject orders.
- Artisan shall be able to update order status (processed, dispatched, etc.).
- Artisan shall be able to track their earnings and sales records.

iii. Requirements of User (Customer):

- User shall be able to register and login into the system.
- User shall be able to browse and search for products.
- User shall be able to view product details and artisan information.
- User shall be able to add products to cart and place orders.
- User shall be able to make payments through available methods
- User shall be able to track order status and delivery updates.

- User shall be able to view previous orders and purchase history.
- User shall be able to provide reviews and ratings for products.

Non Functional Requirement:

- The system should implement secure user authentication and authorization.
- The system should provide real-time updates of order and delivery status.
- The system should support role-based access control.
- The system should be user-friendly and easy to navigate.
- The system should ensure data security and reliability.
- The system should be efficient and responsive.

4.2 Block Diagram of System:

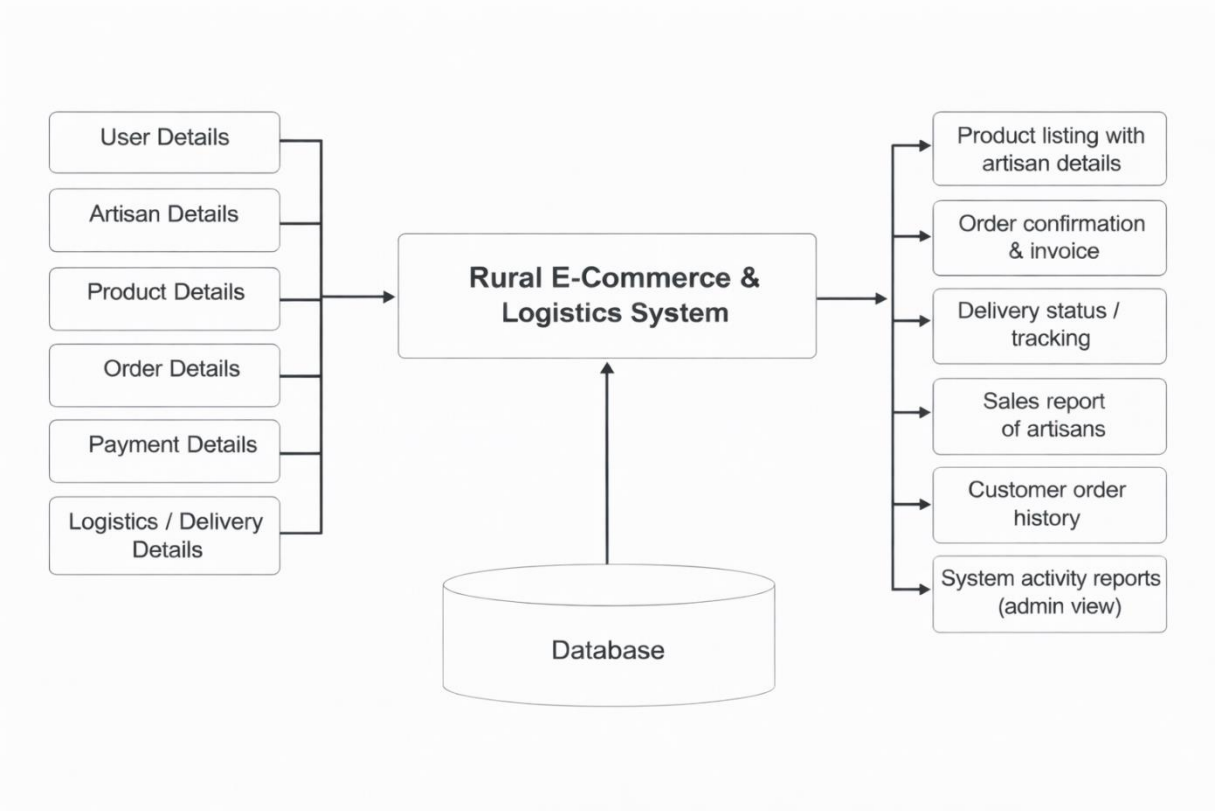

Figure 4.1 Block diagram of Rural E-commerce & Logistics

Figure 4.1 shows the proposed system diagram of Rural E-Commerce & Logistics System, where the diagram is divided into three parts: input required for the system, output provided by the system, and database required to store system data.

Input: User details consist of all relevant data regarding customers such as name, email, mobile number, address, and password. Artisan details include artisan name, contact information, product category, and login credentials. Product details contain product name, description, price, quantity, and images. Order details include order ID, product details, quantity, total amount, and order status. Payment details consist of payment mode, transaction ID, and payment status. Logistics details include delivery address, shipment status, and tracking information.

Output: The system will provide product listings along with artisan details for users. The system will generate order confirmation and invoice for customers. It will provide delivery status and tracking information for each order. The system will generate sales reports for artisans and order history for customers. It also provides system activity reports for admin and ensures easy monitoring and management of all activities.

4.3 Action Plan:

Steps followed to developed Rural E-commerce & Logistics system:

- Finalization of project team and understanding project requirements
- Identification of project domain (Rural E-Commerce & Logistics)
- Study of existing systems and literature survey
- Problem identification and system specification
- Designing system architecture and modules (Admin, Artisan, User)
- Preparation of project planning and documentation
- Design User Interface (UI/UX)
- Design database structure and collections
- Development of frontend using React.js
- Backend development using Node.js
- Database integration using MongoDB
- Implementation of product, order, and logistics modules
- Testing of system functionalities
- Debugging and modifications based on feedback
- Preparation of project presentation and report
- Final submission of project and documentation

4.4. Technology use to develop project:

- Structure: HTML
- Design: CSS
- Validation: JAVASCRIPT
- Backend: Backend
- Database: MongoDB

CHAPTER 5
DETAILS OF DESIGNS, WORKING AND PROCESSES

5.1 System Flow Diagram:

This chapter provides a detailed explanation of the system design, workflow, and working process of the Rural E-Commerce & Logistics System. It explains how different modules of the system such as Admin, Artisan, and User work together and how data flows within the system. The chapter includes system flow, module-wise functionality, database structure, and the complete working process from product listing to order placement, payment processing, and delivery tracking. The purpose of this chapter is to give a clear understanding of how the system operates internally and ensures smooth functioning of the overall application.

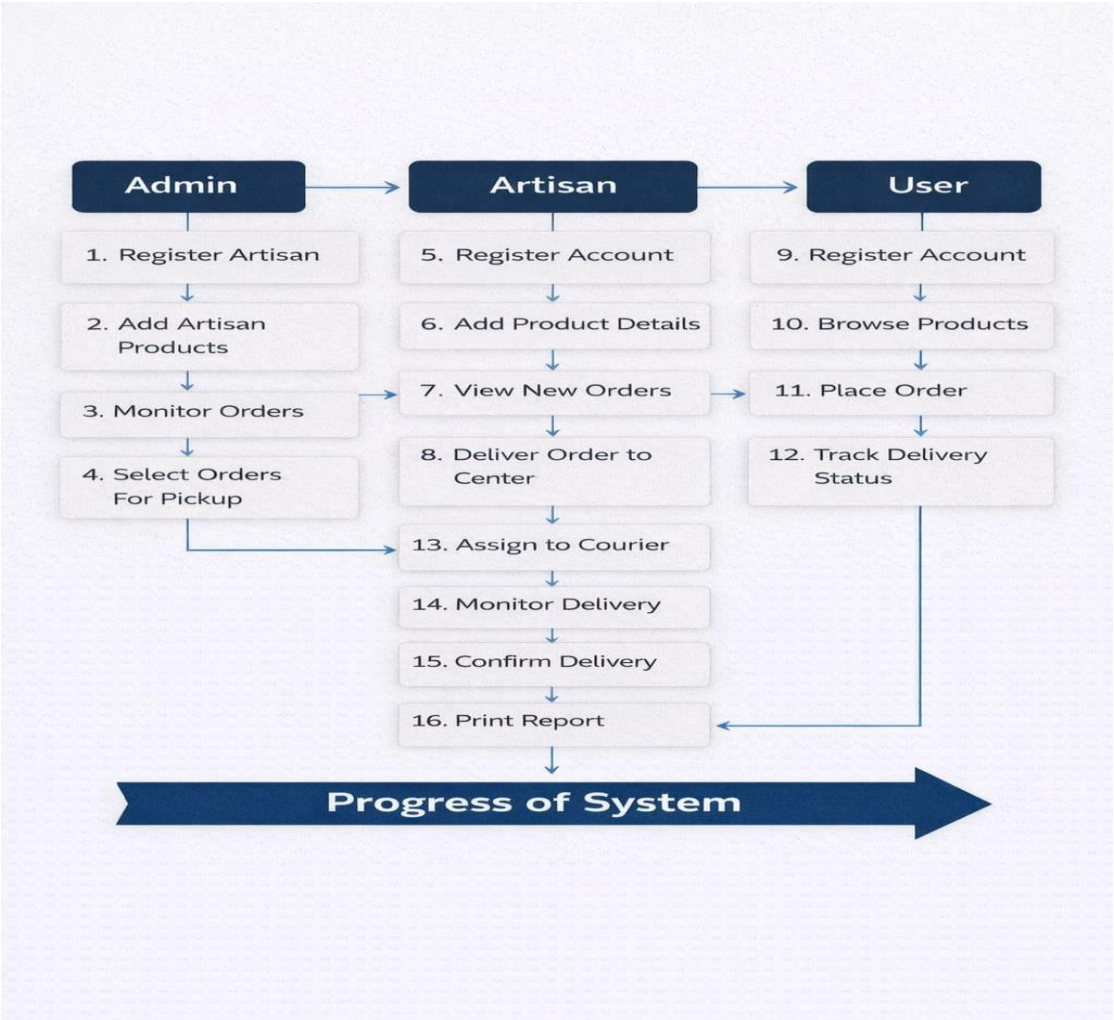

Figure 5.1 Flow diagram of Rural E-commerce & Logistics

This diagram represents the complete working process of the Rural E-Commerce & Logistics System, starting from product browsing to final delivery and status update. The system is divided into four main parts: User (Customer), System Actions, Artisan/Admin, and Logistics/Delivery, all connected through a central database.

1. User (Customer Module)

The process starts with the user.
First, the user registers or logs into the system. After successful login, the user browses products available on the platform.
The user selects products and places an order by entering required details such as:
• Product selection
• Quantity
• Delivery address
• Payment method

After filling in the details, the user submits the order. Once the order is placed, the user can view order status anytime to track progress.

2. System Actions (Automated Process)

After the user places the order, the system automatically performs several actions:
• Receives the order request from the user
• Stores product and order details securely
• Saves customer and transaction data
• Processes payment information
• Updates order status based on progress
• Stores all information in the database

This process is automated and ensures smooth and efficient system operation.

3. Artisan / Admin Module

Once the order is stored, the artisan and admin can access it.
• Artisan views product orders received
• Artisan processes and prepares products for delivery
• Admin monitors all system activities

• Admin manages users, products, and orders

• Admin can approve/reject products and manage overall workflow

5.2 Admin Module Flow:

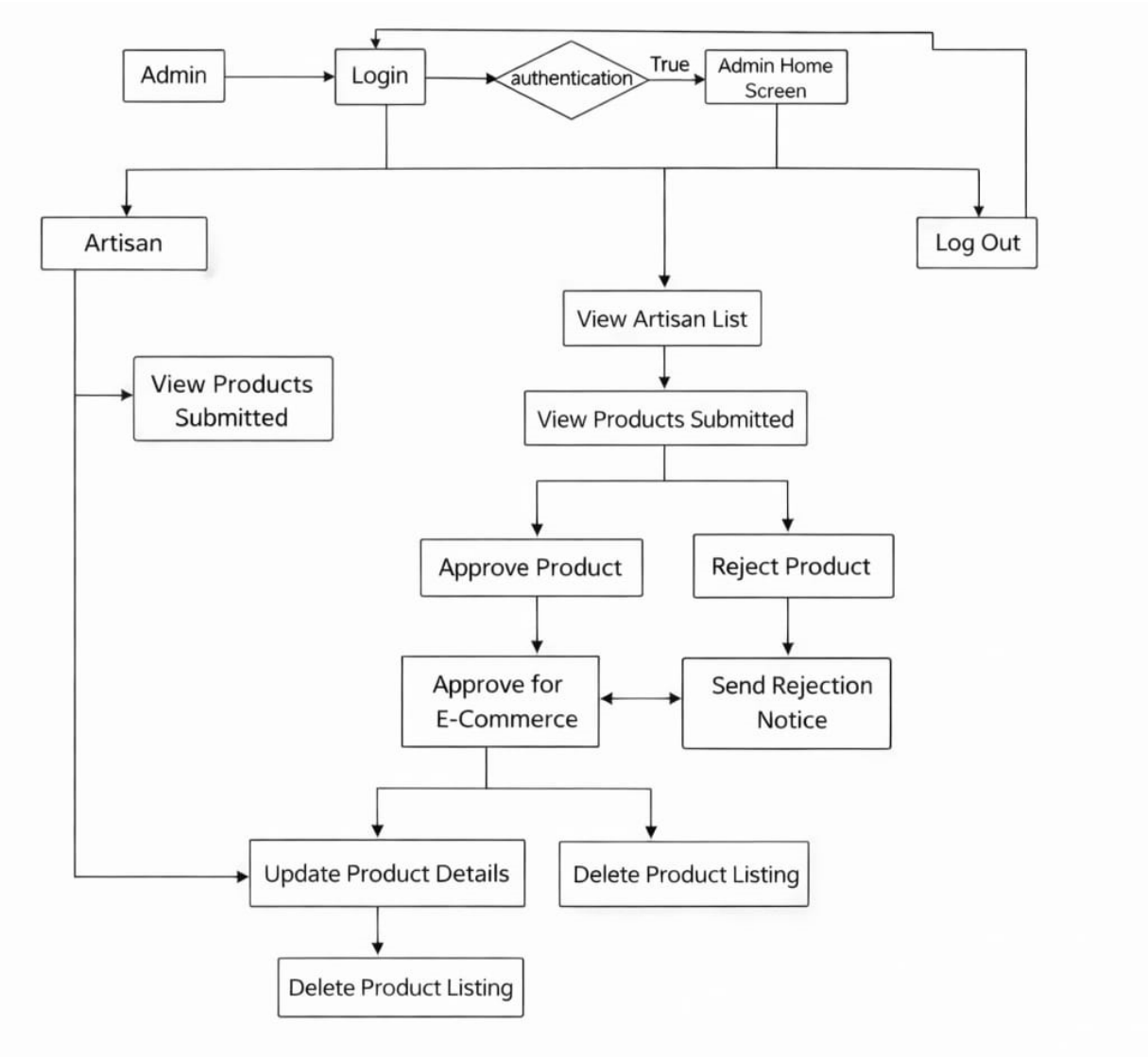

Figure 5.2 Admin module flow

This diagram explains the complete flow of how the admin interacts with the Rural E-Commerce & Logistics System. It shows each step starting from admin login to managing artisans, products, orders, and logistics. The process is designed to be efficient, secure, and easy to manage.

1. Admin Login & Authentication

The process begins with admin login.

The admin enters valid credentials such as email and password to access the system.

• If the credentials are correct, access is granted

• Otherwise, login is denied

This step ensures secure access to the system and restricts unauthorized users.

2. Access Dashboard

After successful login, the admin is redirected to the dashboard.

• The dashboard acts as the main control panel

• It displays summary details like total users, artisans, products, and orders

• It provides navigation to different modules

This helps the admin to monitor overall system performance easily.

3. Manage Artisan

From the dashboard, the admin selects Artisan Management.

• Admin can view all registered artisans

• Admin can approve or reject artisan applications

• Admin can update or delete artisan details

This ensures only valid artisans are allowed to sell products.

4. Manage Products

Admin can manage all products listed in the system.

• View products added by artisans

• Approve or reject products

• Update or delete product details

This step maintains product quality and platform standards.

5. Manage Orders

Admin can monitor all orders placed by users.

• View order details such as customer, product, and quantity

• Assign orders for delivery

• Update order status

This ensures smooth order processing and tracking.

6. Manage Logistics / Delivery

Admin handles delivery-related activities.

• Assign delivery for orders

• Track delivery status (dispatched, in transit, delivered)

• Ensure timely delivery

This connects order processing with final delivery.

7. System Monitoring & Reports

Admin can monitor overall system performance.

• View reports like total sales, orders, and users

• Analyze system activity

• Make decisions based on data

8. Logout

After completing all tasks, the admin logs out of the system.

This ensures system security and proper session management.

5.3 Artisan Module Flow:

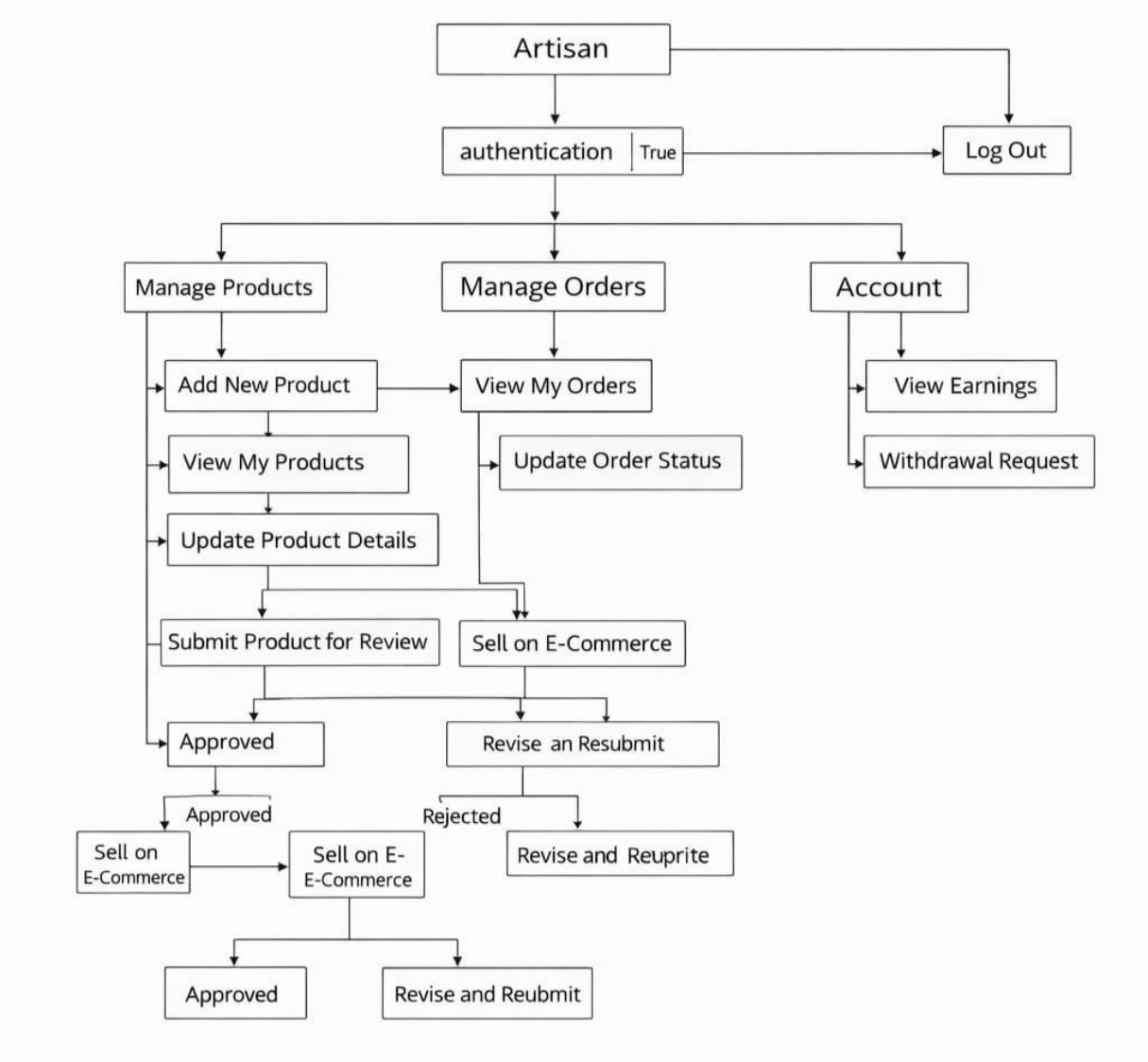

Figure 5.3 Artisan module flow

This diagram explains the working process of the Artisan module in the Rural E-Commerce & Logistics System. It shows how the artisan manages products and orders from login to updating order status.

1. Artisan Login

The artisan logs into the system by entering email and password.

The system verifies the credentials and allows access only if they are correct. This step ensures security and authorized access to the system.

2. Open Artisan Dashboard

After login, the artisan is redirected to the dashboard.
The dashboard acts as the main page where the artisan can view an overview of activities such as total products, orders, and earnings. It also provides navigation to different sections like products and orders.

3. Manage Products

The artisan can manage their products in the system.
• Add new products with details such as name, price, description, and image
• View all added products
• Update or delete product details

This helps the artisan maintain and control their product listings.

4. Submit Products for Approval

After adding products, the artisan submits them for admin approval.
• Admin reviews the product details
• Products are either approved or rejected

Approved products are displayed on the platform, while rejected products can be updated and resubmitted.

5. View Orders

The artisan can view all orders received for their products.
Each order contains details such as customer information, product details, and quantity.

6. Update Order Status

The artisan processes the orders and updates their status.
• Status can be updated as processed, shipped, or completed
• This status is visible to both admin and customer

This ensures proper tracking and order handling.

7. Logout

After completing all tasks, the artisan logs out of the system.

This ensures system security and proper session handling.

5.4 User (Customer) Module Flow:

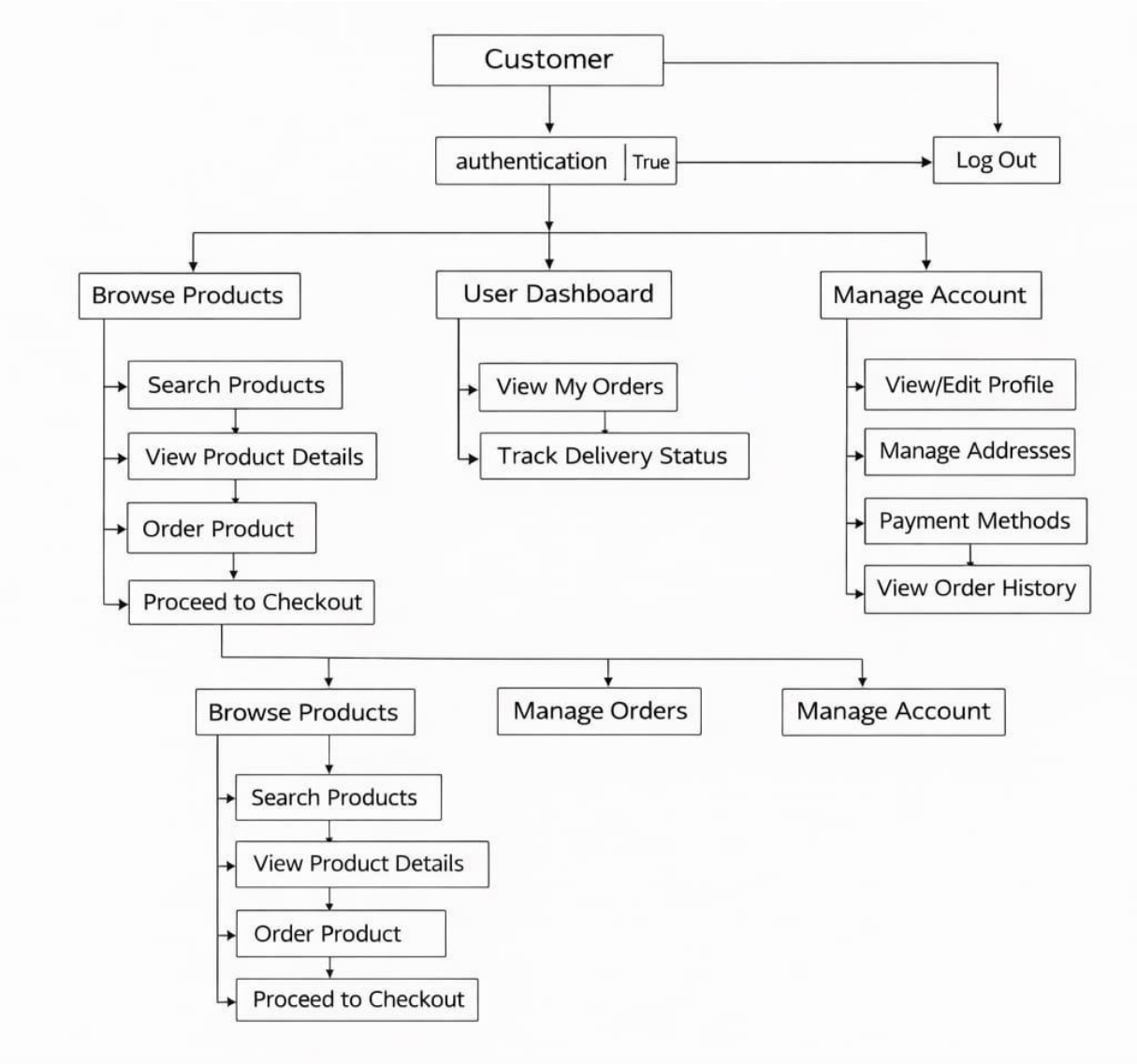

Figure 5.4 User (Customer) module flow

This diagram explains the working process of the User (Customer) module in the Rural E-Commerce & Logistics System. It shows how the user interacts with the system from login to placing orders and tracking them.

1. User Login / Registration

The process begins with user login or registration.

• If the user is new, they need to register by providing details like name, email, mobile number, address, and password

• If already registered, the user can log in using credentials

This step ensures only authorized users can access the system.

2. Open User Dashboard

After successful login, the user is redirected to the dashboard.

The dashboard acts as the main interface where the user can browse products, view orders, and manage account details.

3. Browse Products

The user can explore various products available on the platform.

• View product details such as name, price, and description

• Select products based on interest

This helps the user to choose suitable products easily.

4. Add to Cart & Place Order

The user selects products and adds them to the cart.

• User can update quantity or remove items

• Proceed to checkout and enter delivery details

• Select payment method and place order

5. Payment Process

The user completes the payment process.

• Enter payment details

• Confirm transaction

• Order is successfully placed after payment

6. Order Status

After placing the order, the user can track order status.

• Status like pending, processed, shipped, delivered

• Real-time updates are provided

This keeps the user informed about order progress.

7. Logout

After completing all activities, the user logs out of the system.

This ensures security and proper session management.

5.5 ER Diagram:

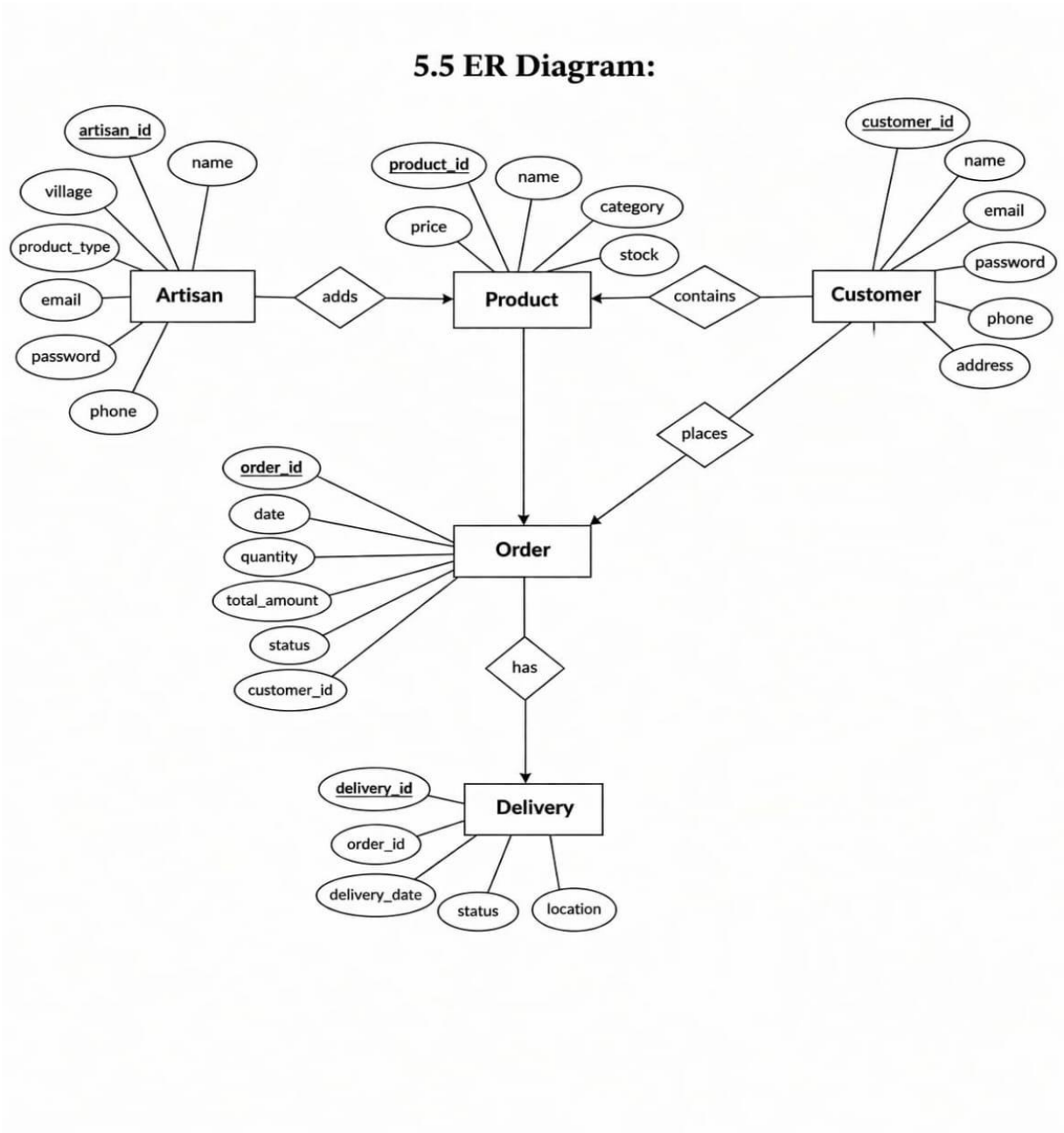

Figure 5.5 E-R diagram of Rural E-commerce & Logistics

Figure 5.5 shows the overview of the database for the Rural E-Commerce & Logistics System, where entities represented in rectangles are strong entities such as Admin, Artisan, User, Product, Order, and Payment, which have their own tables in the database. The weak entities, shown in diamond shapes, represent relational tables such as Order Details, Product Review, and Delivery Status, as they depend on strong entities for their existence. The ovals represent attributes of each entity, which correspond to the field names in the database.

Table 2 Database table description

Sr. No.	Table Name	Fields Name	Purpose
1	User	Id, first name, last name, email, mobile no, address, password	User table contains customer personal information for login and ordering products.
2	Artisan	Id, first name, last name, mobile no, email, address, category, password	Artisan table contains rural seller details and product category information.
3	Product	Id, artisan id, product name, description, price, quantity, image	Product table contains all product-related information uploaded by artisans.
4	Order	Id, user id, product id, quantity, total amount, order date, status	Order table stores order details placed by users and their status.
5	Payment	Id, order id, payment mode, transaction id, amount, payment status	Payment table contains transaction and payment details of orders.
6	Cart	Id, user id, product id, quantity	Cart table stores products added by user before placing an order.

Functionality provided by System:

1. User Management
2. Artisan Management
3. Product Management
4. Order Management
5. Payment Tracking

1. User Management:

The User Management module of the system handles information about all users interacting with the system such as Admin, Artisan, and User (Customer). Each user is assigned specific roles and permissions so that they can access only relevant functionalities. Admin manages all users, artisans manage their products, and users browse and purchase products. This module ensures proper organization, security, and easy management of user data.

Step wise procedure of user management module:

Step 1. Registration of User and Artisan through the form

- User needs to fill details such as: Name, Email, Mobile Number, Address, Password.
- Artisan needs to fill details such as: Name, Mobile Number, Email, Address, Product Category, Password.

Step 2. Admin can view all User and Artisan records on the dashboard.

Step 3. Admin can apply filters and sorting (ascending/descending) to easily search and manage users.

Step 4. Admin can update or delete User and Artisan records using provided options.

Step 5. Admin can manage user access and monitor system activities.

Implementation of User Management Module:

Algorithm:

Step 1: Start

Step 2: Create User table in database

Step 3: Create Artisan table in database

Step 4: Add User record through registration form

Step 5: Add Artisan record through registration form

Step 6: Display all User and Artisan records in table format

Step 7: Add update and delete options for each record

Step 8: If update action is clicked, update record using form data

Step 9: If delete action is clicked, confirm and delete record

Step 10: Repeat steps for both User and Artisan

Step 11: End

2. Artisan Management:

After registration of artisans, the admin is empowered to manage artisan details effectively. This module handles all artisan-related information such as personal details, product categories, and account status. Admin can view, update, and remove artisan records, while artisans can manage their profiles and products. This module ensures proper coordination between artisans and the system, helping in smooth product listing and management.

Step wise procedure of Artisan Management module:

Step 1. Artisan registers into the system by filling details such as Name, Email, Mobile Number, Address, Category, and Password.

Step 2. After registration, artisan details are stored in the system and visible to admin on dashboard.

Step 3. Admin can view all artisan records in tabular format.

Step 4. Admin can apply filters and sorting to easily search artisans.

Step 5. Admin can update or delete artisan details using provided options.

Step 6. Artisan can login and manage their profile details.

Implementation of Artisan management module:

Algorithm:

Step 1: Start

Step 2: Create Artisan table in database

Step 3: Add artisan record through registration form

Step 4: Display artisan records in table format

Step 5: Provide update and delete options

Step 6: If update action is clicked, update artisan details

Step 7: If delete action is clicked, confirm and delete record

Step 8: Allow artisan to login and manage profile

Step 9: End

3. Product Management:

After registration of artisans, they are empowered to add and manage their products in the system. This module handles all product-related information such as product name, description, price, quantity, and images. It ensures that products are properly listed and available for users to browse and purchase. Admin can monitor all products, while artisans can update or delete their own products. This module acts as a central hub for managing product data efficiently.

Stepwise procedure of Product Management Module:

Step 1. After artisan login, the dashboard displays options to manage products.

Step 2. Artisan can add new product by filling details such as product name, description, price, quantity, and image.

Step 3. Product details are stored in the system and become visible to users.

Step 4. Artisan can view all their products in a list format.

Step 5. Artisan can update or delete product details anytime.

Step 6. Admin can view all products and monitor product activities.

Implementation of Task Management:

Algorithm for adding product:

Step 1: Start

Step 2: Login as Artisan

Step 3: Open product form

Step 4: Enter product details (name, description, price, quantity, image)

Step 5: Store product data in database

Step 6: Display product in product list

Step 7: End

Algorithm for product management:

Step 1: Start

Step 2: Fetch product records from database

Step 3: Display products in table format

Step 4: Provide update and delete options

Step 5: If update action is clicked, update product details

Step 6: If delete action is clicked, delete product from database

Step 7: End

4. Order Management:

The Order Management module handles all activities related to placing, processing, and tracking orders in the system. Users can browse products and place orders, while artisans receive and process those orders. Admin monitors all order activities and ensures smooth execution of the process. This module manages order details such as product, quantity, total amount, and order status, ensuring transparency and efficient handling of orders.

Stepwise procedure of Order Management Module:

Step 1. User logs into the system and browses available products.

Step 2. User selects product and places order by providing required details.

Step 3. Order details are stored in the system with unique order ID.

Step 4. Artisan can view new orders received on their dashboard.

Step 5. Artisan accepts and processes the order.

Step 6. Order status is updated (e.g., placed, processed, dispatched).

Step 7. Admin can view and monitor all orders in the system.

Implementation of Timeline Management:

Algorithm:

Step 1: Start
Step 2: User selects product and places order
Step 3: Generate unique order ID
Step 4: Store order details in database
Step 5: Display order details to artisan
Step 6: Artisan updates order status
Step 7: Admin monitors order records
Step 8: End

5. Payment Processing:

The Payment Processing module handles all financial transactions between users and the system. It ensures secure and reliable payment methods for purchasing products. Users can make payments through available options, and the system stores transaction details such as payment mode, transaction ID, amount, and status. This module helps in maintaining transparency and accurate financial records, while admin can monitor all payment activities.

Stepwise procedure of Payment Processing Module:

Step 1. User selects product and proceeds to checkout.

Step 2. User selects payment method (online payment / cash on delivery).

Step 3. User enters required payment details.

Step 4. Payment is processed and transaction is completed.

Step 5. Payment details are stored in the system with transaction ID.

Step 6. Order status is updated based on payment status.

Step 7. Admin can view and monitor all payment transactions.

Implementation of Payment Processing Module:

Algorithm:

Step 1: Start

Step 2: User selects product and proceeds to payment

Step 3: Select payment mode

Step 4: Process payment transaction

Step 5: Generate transaction ID

Step 6: Store payment details in database

Step 7: Update order status

Step 8: End

CHAPTER 6
RESULT AND APPLICATION

6.1 Result:

The system developed was tested for our Rural E-Commerce & Logistics System project by final year Diploma in Computer Engineering students. The web pages were verified for proper functionality including user, artisan, and admin modules, and the results obtained are for the current year.

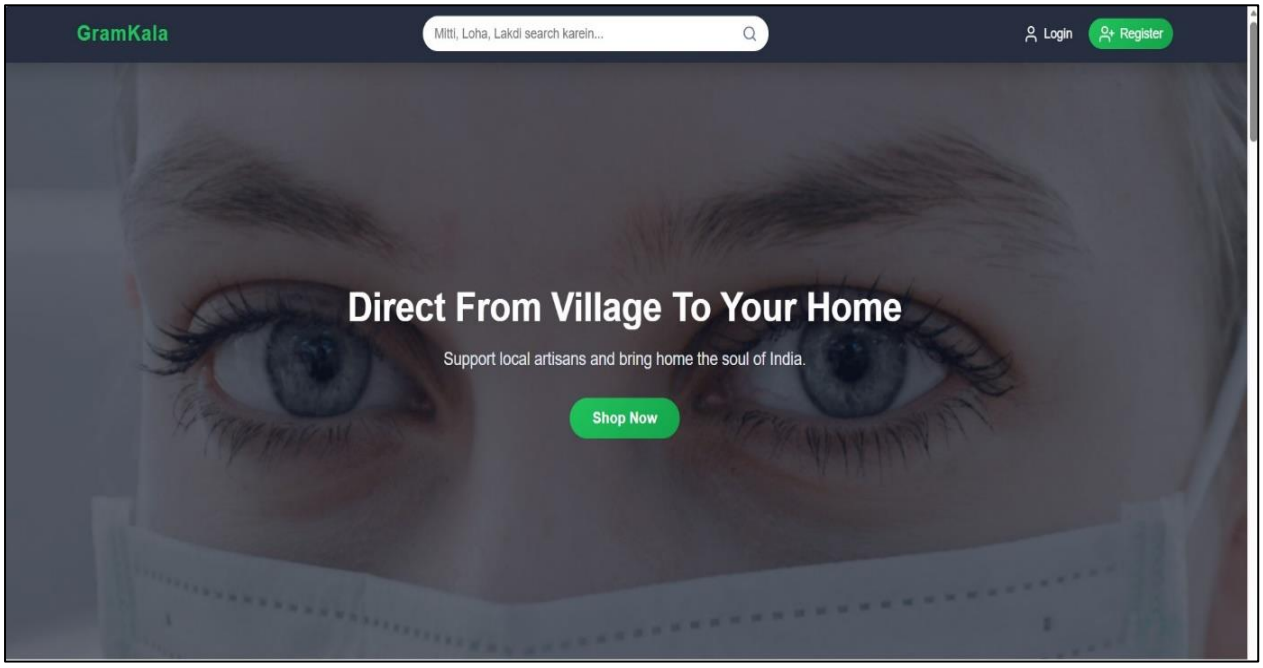

Figure 6.1 Home Page of Rural E-commerce & Logistics

These web page is the landing page of our Rural E-Commerce & Logistics *System* for all users, where users need to select options as per their role. For example, Customer selects the User option, Artisan selects the Artisan option, and Admin selects the Admin option, shown on the main section of the webpage. The homepage also provides basic navigation such as browsing products and login/register options. The system displays relevant information and allows users to access their respective modules easily, ensuring smooth interaction with the website.

Registration Form Web Page for Admin:

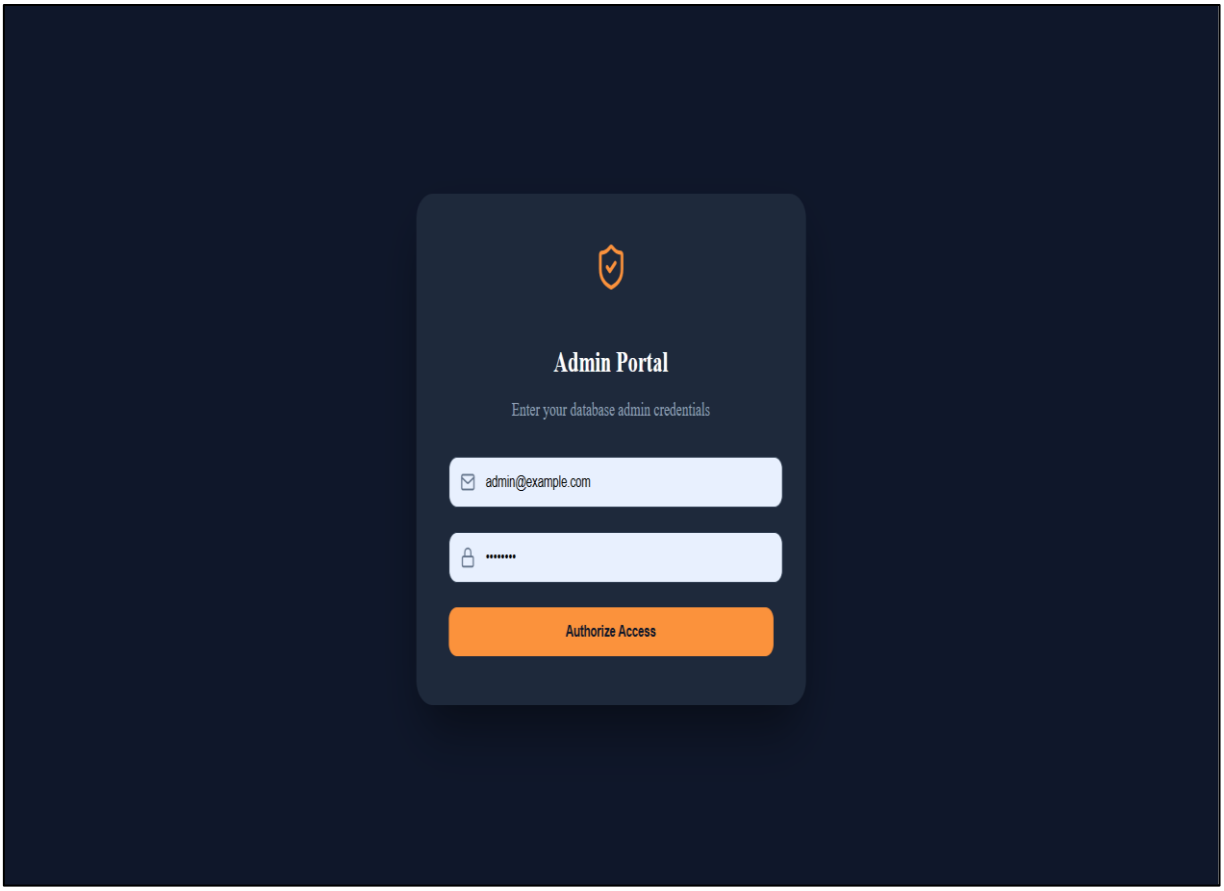

Figure 6.2 Registration form for Admin

Admin data is entered through the above login form in the system, where the admin needs to provide valid credentials such as email and password. After entering the required details, the admin clicks on the authorize access button to log into the system. This screen ensures secure authentication before accessing the admin dashboard.

Components Used:

• Input fields (Full Name, Email, Password)

• Register button

• Login link

• Card layout

```
const handleSubmit = async (e) => {
e.preventDefault(); const
validationError = validate(); if
(validationError) {
```

```
setError(validationError);   return; } try
{
  await API.post("/auth/register", form);   navigate("/"); } catch (err) {
setError(err.response?.data?.message || "Registration failed");
}
};
```

Table 3: Registration Validations

Fields	Validation Applied
Full Name	"^[A-Za-z]{3,30}$", "Name must contain only alphabets and spaces (3–30 characters)"
Email	"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+.[A-Zaz]{2,}$", "User must enter valid email format
Password	"(?=.\d)(?=.[a-z])(?=.*[A-Z]).{8,16}", "Password must contain at least one number, one uppercase letter, one lowercase letter, and length between 8 to 16 characters"
Empty Fields Validation	"All fields (Name, Email, Phone, Password) must not be empty"
Invalid Email	"Display error message if email format is incorrect"

Registration Form Web Page for Artisan:

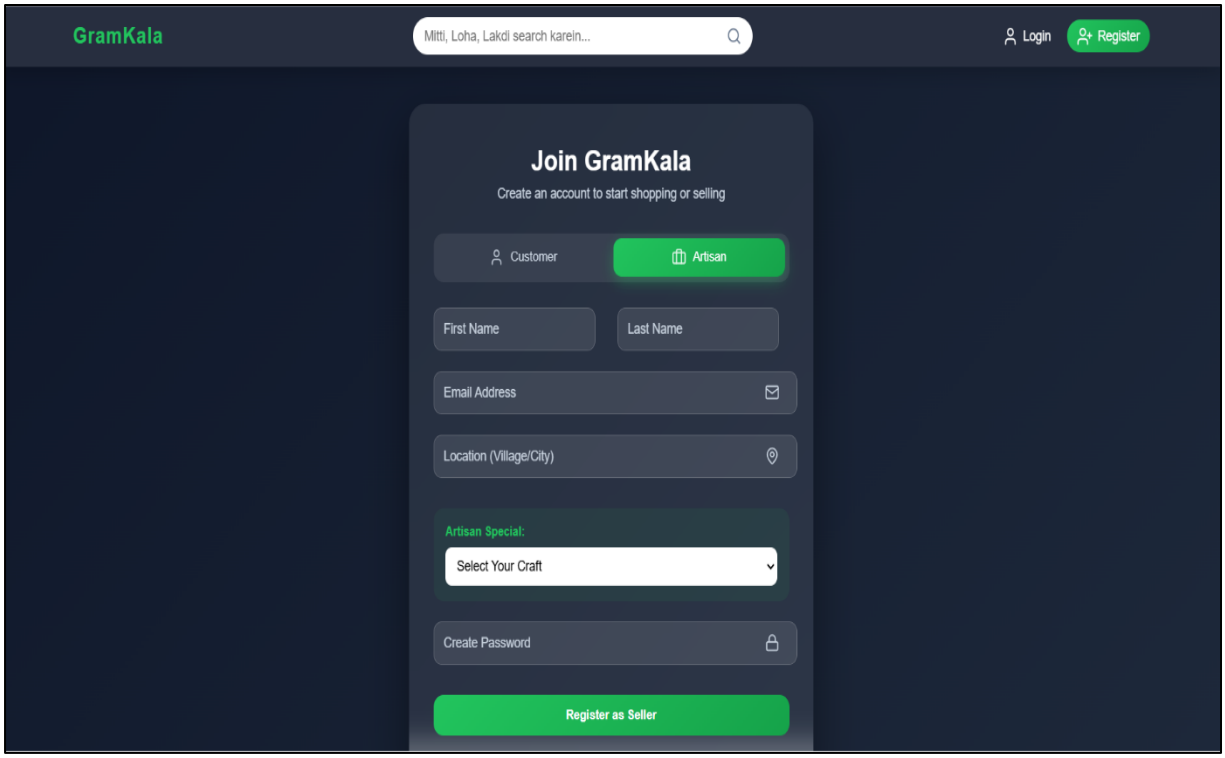

Figure 6.3: Customer Login Page

User data is entered through the above registration form in the system, where the user needs to fill required details such as first name, last name, email address, location, and password. If the user selects the artisan option, they must also choose their craft or specialization. After filling all the details, the user clicks on the register button to create an account. This screen allows new users to register and access the system based on their role.

Components Used:

- Input fields (Email, Password)
- Buttons (Login, Back)
- Register link
- Card layout

```
const handleSubmit=async(e)=>{
e.preventDefault();   try{
const res = await API.post("/auth/login",form);
localStorage.setItem("token",res.data.token);
localStorage.setItem("role",res.data.user.role);
```

```
localStorage.setItem("name",res.data.user.name);
navigate("/customer/dashboard");
}catch(err){
setError(err.response?.data?.message || "Login failed");
```

Registration Form Web Page for Customer:

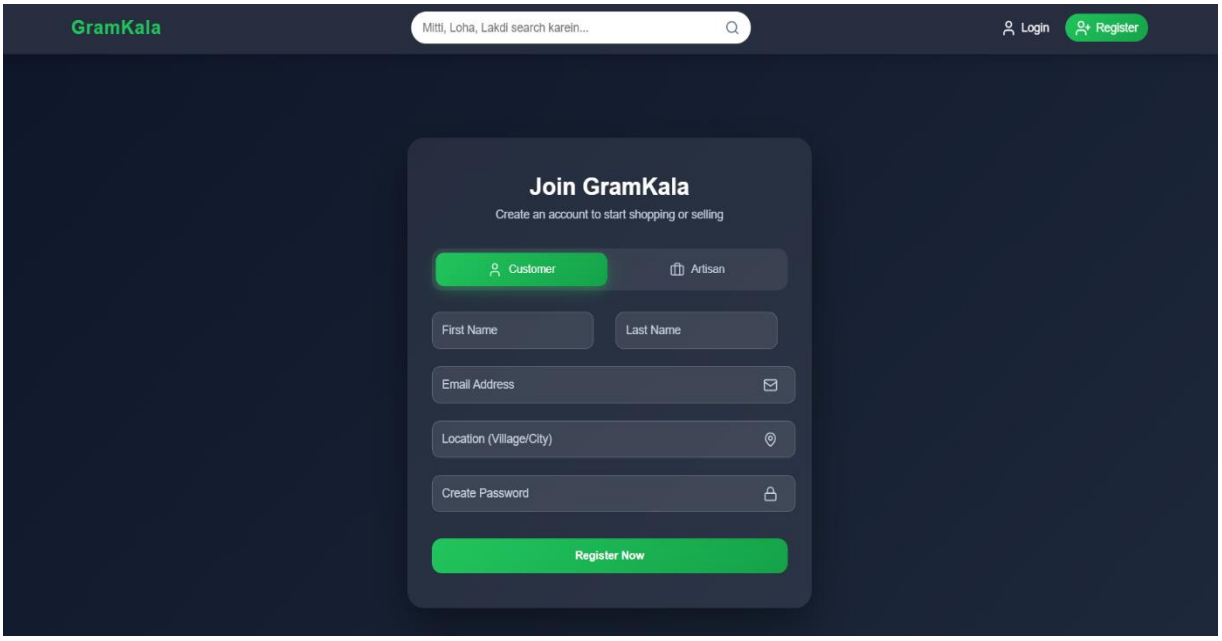

Figure 6.4 Registration form for Customer

Customer data is entered through the above registration form in the system, where the user selects the Customer option and fills required details such as first name, last name, email address, location, and password. After entering all the necessary information, the user clicks on the register button to create an account. This screen enables customers to register and access the system for browsing products and placing orders.

Components Used:

• Input fields (Full Name, Email, Password)

• Register button

• Login link

• Card layout

```
const handleSubmit = async (e) => {
```

```
e.preventDefault(); const
validationError = validate(); if
(validationError) {
setError(validationError);   return; } try
{
  await API.post("/auth/register", form);   navigate("/"); } catch (err) {
setError(err.response?.data?.message || "Registration failed");
}
};
```

Login Page:

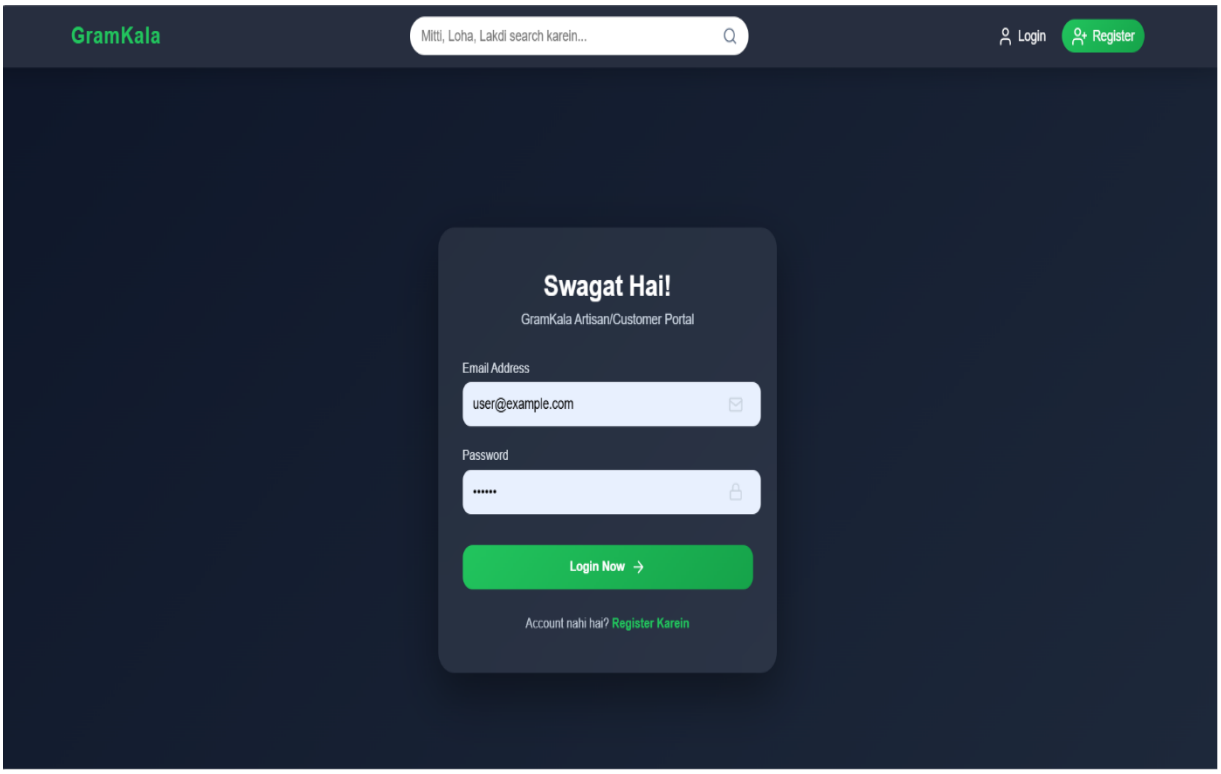

Figure 6.5 Login Page for Artisan/Customer

Login page of artisan/customer accepts artisan/customer email and password to allow only authorized user to access admin dashboard. The login button allows access to the supplier dashboard, while the back button navigates to the previous page.

Components Used:

- Input fields (Email, Password)
- Buttons (Login, Back)
- Centered card layout

Admin Panel:

1. Dashboard:

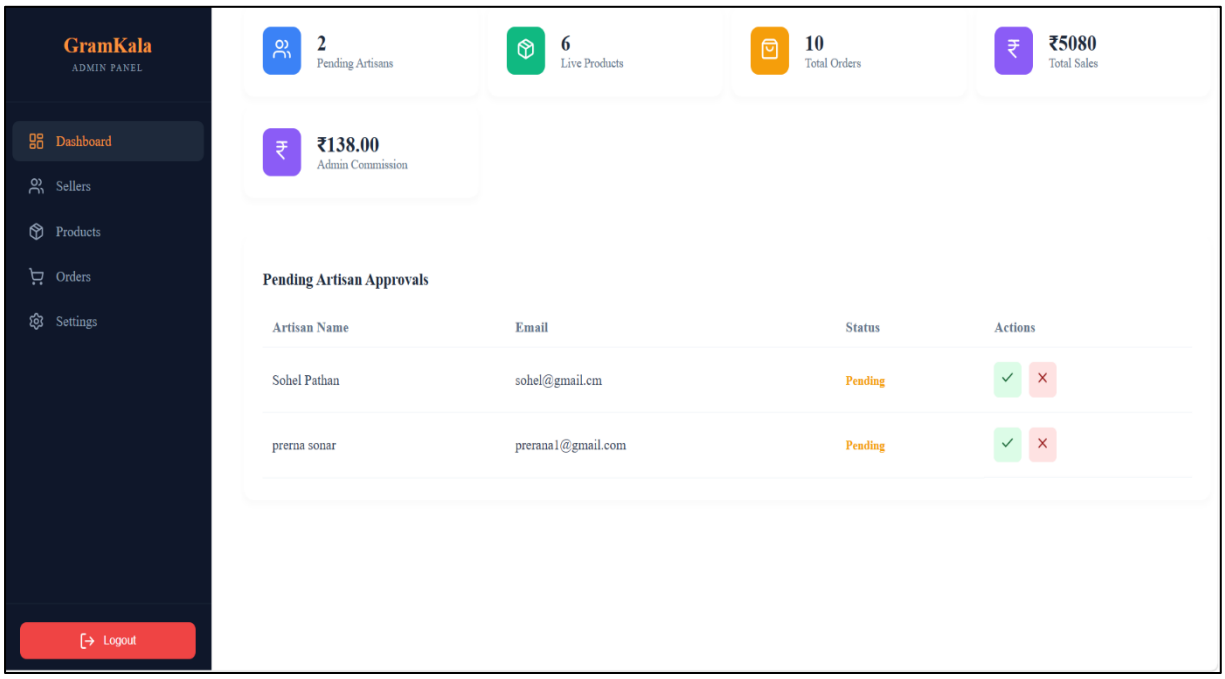

Figure 6.6 Admin Panel Dashboard

The Admin dashboard contains a sidebar menu, header section, and main content area. The sidebar is placed on the left side and includes options such as Dashboard, Sellers, Products, Orders, Settings, and Logout. The dashboard provides an overview of system activities and helps the admin manage the platform efficiently.

The top section displays summary cards which show important system data such as:

• Total number of pending artisans

• Total live products available on the platform

• Total number of orders placed

• Total sales amount

• Admin commission earned

Below the summary section, the dashboard displays a table for Pending Artisan Approvals**.** This table shows details such as artisan name, email, and current status. The admin can take actions directly from this table using approve or reject buttons.

Admin can perform the following actions:

• View overall system statistics from dashboard cards

• Approve or reject artisan registration requests

• Manage sellers, products, and orders from sidebar menu

• Monitor sales and commission details

• Logout securely from the system

2. Sellers:

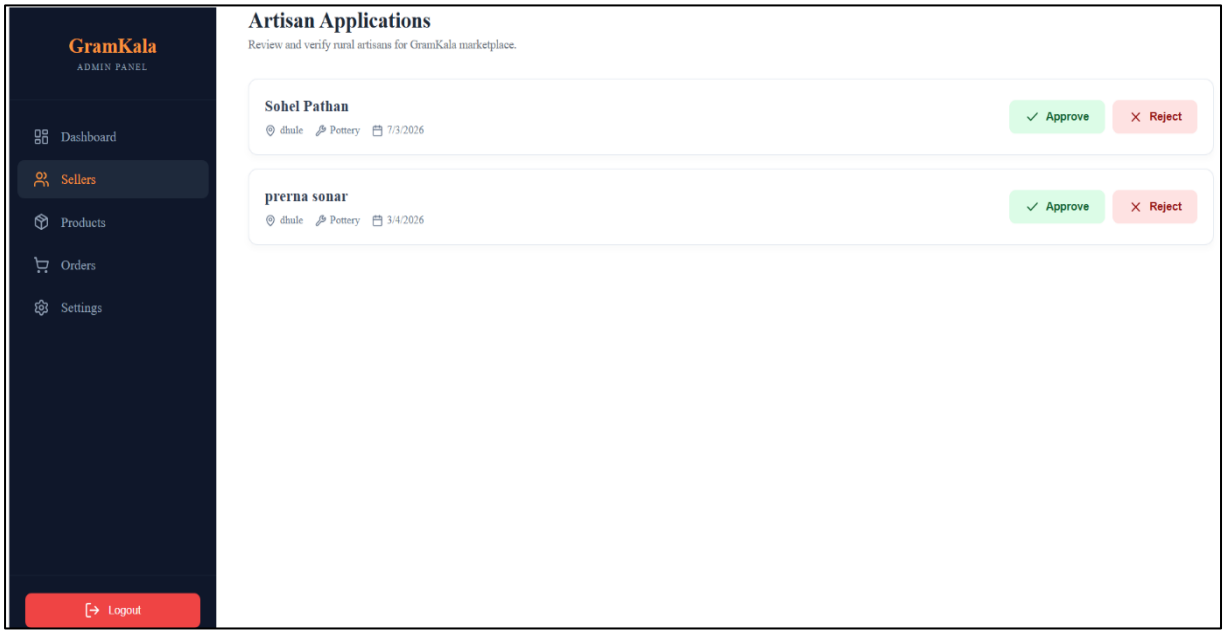

Figure 6.7 Admin Panel Sellers Page

This page allows the admin to review and manage artisan registration requests in the system. It contains a sidebar menu on the left side with options such as Dashboard, Sellers, Products, Orders, Settings, and Logout. The main section displays a list of artisan applications submitted for approval.

Each artisan entry shows important details such as artisan name, location, category (e.g., pottery), and registration date. The admin can easily verify the details of each artisan before taking action.

The page provides action buttons for each artisan application:

• **Approve** – to accept the artisan and allow access to the platform

• **Reject** – to decline the application

Admin can perform the following actions:

- View all artisan registration requests
- Verify artisan details before approval
- Approve valid artisan applications
- Reject invalid or incomplete applications
- Manage sellers from the sidebar menu

3. Products:

Figure 6.8 Admin Panel Products Page

This page allows the admin to view and manage all products available in the system. It includes a sidebar menu on the left side with options such as Dashboard, Sellers, Products, Orders, Settings, and Logout. The Products section displays a structured table containing all product details.

The table shows important information such as product ID, product name, category, price, seller name, available stock, and product status. The status indicates whether the product is approved and available for customers. A search bar is also provided at the top to quickly find products or sellers.

Each product entry includes action buttons that allow the admin to update or delete product details. This helps in maintaining accurate and up-to-date product listings on the platform.

Admin can perform the following actions:

- View all products listed by artisans
- Search products or sellers using search bar
- Check product details like category, price, and stock
- Update product information
- Delete unwanted or invalid product listings
- Monitor product approval status

4. Orders:

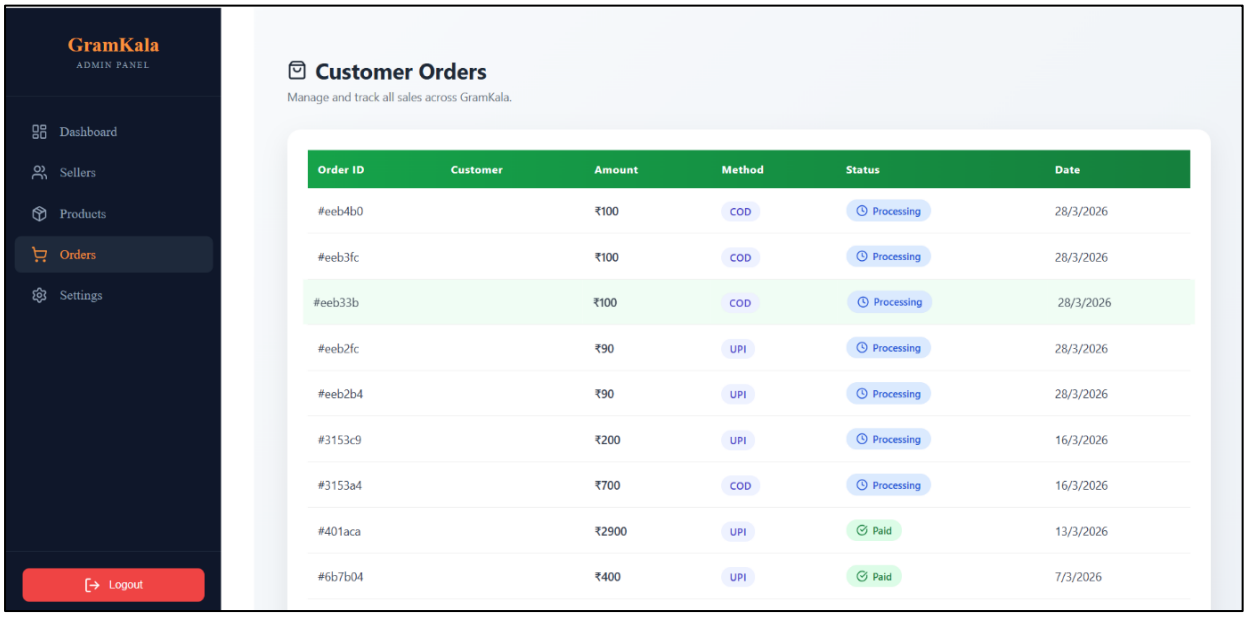

Figure 6.9 Admin Panel Orders Page

This page allows the admin to view and manage all customer orders in the system. It contains a sidebar menu on the left side with options such as Dashboard, Sellers, Products, Orders, Settings, and Logout. The main section displays all customer orders in a structured table format.

The table shows detailed information about each order such as Order ID, Customer name, total amount, payment method, order status, and date. The status column indicates the current progress of the order such as Processing or Paid. This page helps the admin to monitor all transactions and track order activities efficiently.

Admin can perform the following actions:

• View all customer orders in one place
• Check order details like amount, payment method, and date
• Monitor order status (Processing, Paid)
• Manage overall order flow in the system

```
const handleSubmit = async (e) => {
e.preventDefault();
console.log("Submit clicked");
if (!validateForm()) return;
try {
console.log("Calling API...");
const res = await API.post("/orders/create", {
...form,
totalAmount: calculateTotal(),
});
console.log("API RESPONSE:", res);
alert("Order placed successfully!");
navigate("/customer/dashboard");
} catch (error) {
console.error("FULL ERROR:", error.response || error);
alert("Error placing order");
}
};
const total = calculateTotal();
const advance = Math.floor(total * 0.2);
```

Table 4 : New Order Page Validations

Field Name	**Validation / Description**
Customer Name	"Required field", Customer name should not be empty
Product Selection	"Required field", User must select at least one product
Quantity	"^[1-9][0-9]*$", Only positive numbers allowed (minimum 1 item)
Delivery Address	"^[A-Za-z0-9 ,.-]{10,100}$", Enter valid address (10–100 characters)
Pincode	"^[0-9]{6}$", Only 6 digit numeric pin code allowed
Payment Method	"Required field", User must select payment method (COD/UPI)
Order Date	"Auto-generated field", Displays current system date
Total Amount	"Auto-calculated field", Calculated based on product price / quantity

Artisan Portal:

1. **Dashboard:**

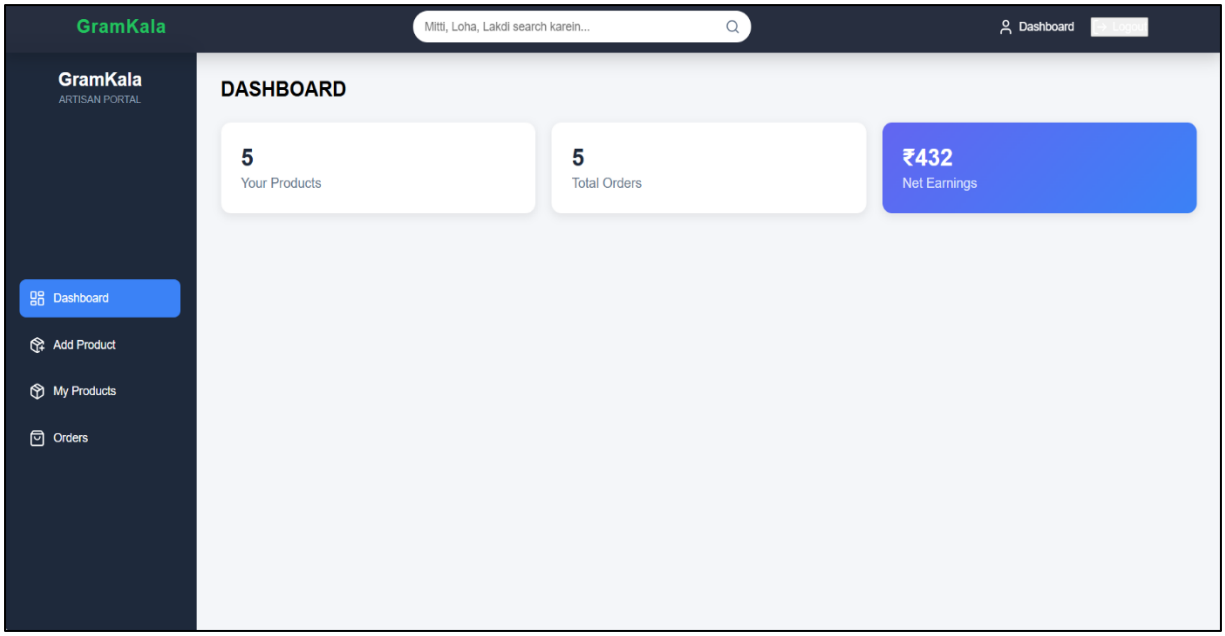

Figure 6.10 Artisan Portal Dashboard

Artisan dashboard contains header, profile section and menu at left side. The menu includes options like Dashboard, Add Product, My Products, Orders and Logout. This page provides an overview of artisan activities on the platform. The top section displays dashboard title along with search bar and navigation options.

The dashboard shows summary cards which represent:

• Total number of products added by artisan

• Total orders received

• Net earnings generated

Artisan can:

• Add new products using “Add Product” option

• View and manage their products

• Check all orders received

• Monitor earnings and performance

• Logout securely from the system

2. Add Product:

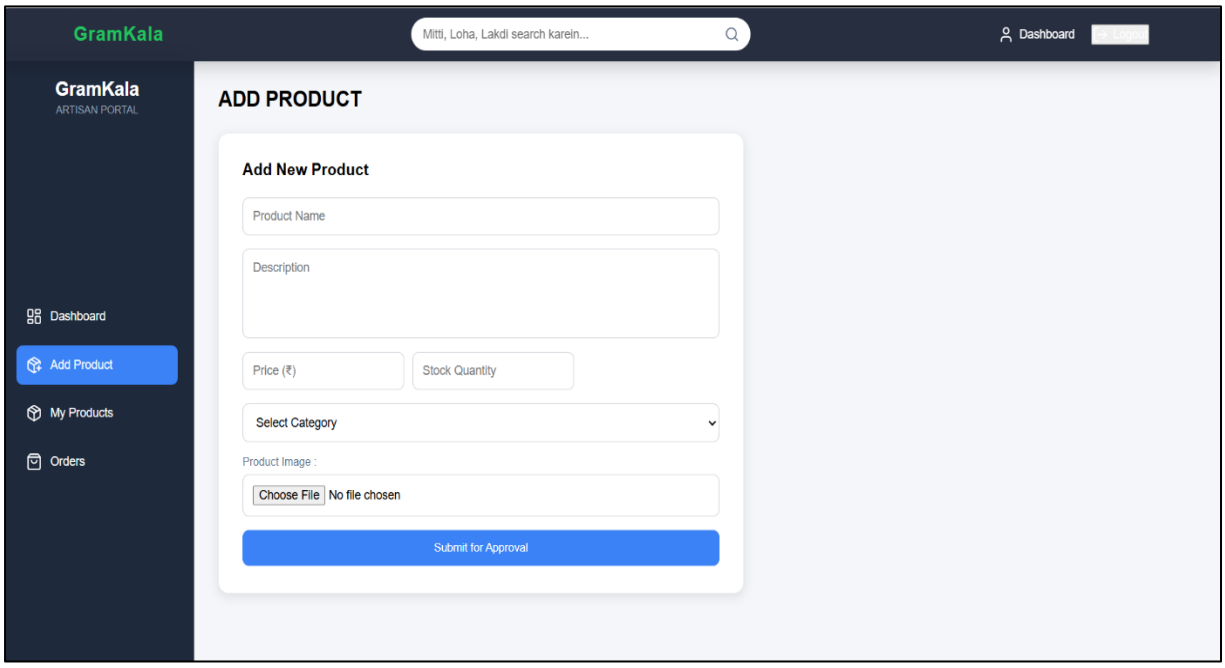

Figure 6.11 Artisan Portal Add Products Page

This page allows the artisan to add new products to the system. It includes a form where the artisan can enter product details such as product name, category, price, stock, description, and upload product image. This page helps artisans to list their products on the platform for customer purchase.

The form ensures that all required fields are filled correctly before submission. After successful submission, the product is added to the system and sent for admin approval.

Artisan can:

- Add new product with complete details
- Upload product image
- Set price and stock quantity
- Submit product for approval
- Manage product listing efficiently

```
const handleAddProduct = async (e) => {
  e.preventDefault();
  if (!validateForm()) return;
```

```
  try {
    const formData = new FormData();
    formData.append("name", form.name);
    formData.append("category", form.category);
    formData.append("price", form.price);
    formData.append("stock", form.stock);
    formData.append("image", form.image);
    const res = await API.post("/products/add", formData, {
      headers: { "Content-Type": "multipart/form-data" }
    });
    alert("Product added successfully!");
    navigate("/artisan/my-products");
  } catch (error) {
    console.error("Error adding product:", error);
    alert("Failed to add product");
  }
};
```

Table 5 :Add Products Validations

Field Name	**Validation / Description**
Product Name	"^[A-Za-z0-9]{3,50}$", Product name must be 3–50 characters
Category	"Required field", User must select product category

Price	"^[1-9][0-9]*$", Only positive numeric values allowed
Stock Quantity	"^[1-9][0-9]*$", Minimum 1 quantity required
Description	"^[A-Za-z0-9 ,.]{10,200}$", Description must be 10–200 characters
Product Image	"Required field", Only image files allowed (jpg, png, jpeg)
Image Size	"Max size 2MB", Image must be within allowed size
Empty Fields Validation	All fields must not be empty

3. My Products:

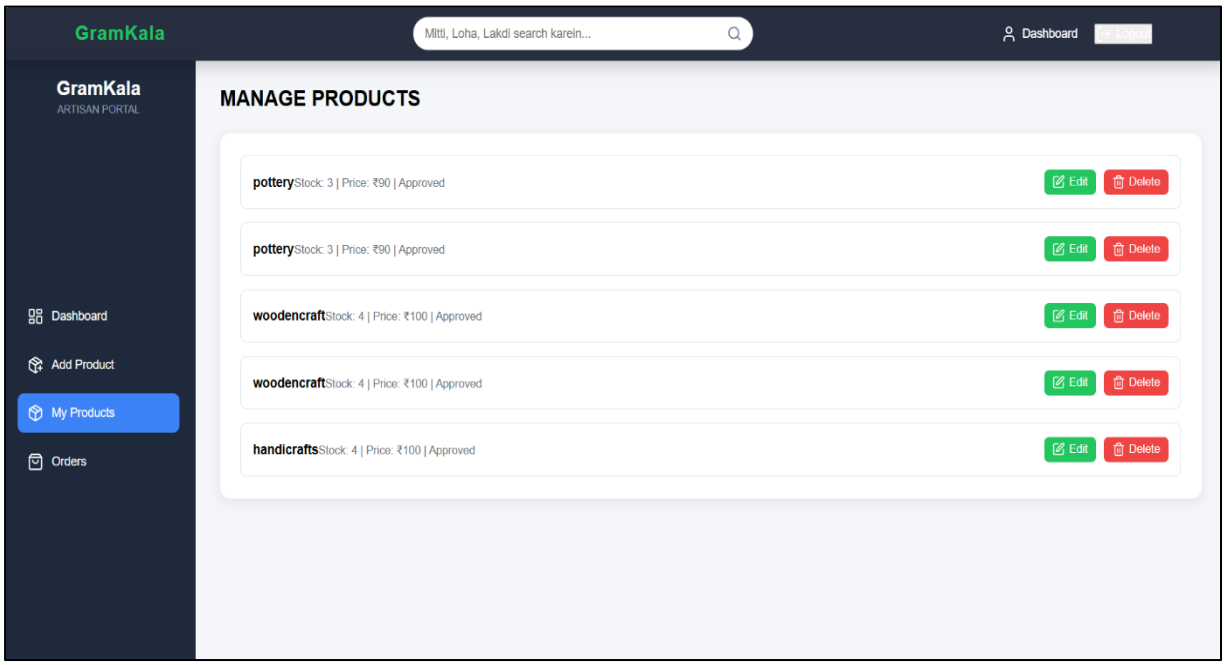

Figure 6.12 Artisan Portal My Products Page

This page displays all products added by the artisan in a structured format for easy management. It allows the artisan to view product details, update information, and manage product listings efficiently.

Each product is shown with details such as product name, category, price, stock quantity, and approval status. The status indicates whether the product is approved, pending, or rejected by the admin.

Artisan can:

• View complete list of their products
• Check product details (name, category, price, stock)
• Monitor product status (Approved, Pending, Rejected)
• Update product details if required
• Delete product from listing
• Manage and track all products efficiently

```
const fetchProducts = async () => {
  try {
    const res = await API.get("/products/my");
    setProducts(res.data || []);
  } catch (error) {
    console.error("Error fetching products:", error);
    alert("Failed to load products");
  } finally {
    setLoading(false);
  }
};
```

Table 6 : My Products Validations

Field Name	**Validation / Description**
Product Name	"^[A-Za-z0-9]{3,50}$", Product name must be 3–50 characters

Category	"Required field", User must select valid category
Price	"^[1-9][0-9]*$", Only positive numeric values allowed
Stock Quantity	"^[0-9]+$", Must be numeric and cannot be negative
Description	"^[A-Za-z0-9 ,.]{10,200}$", Description must be 10–200 characters
Product Image (Update)	Optional, but if uploaded must be valid image format (jpg, png, jpeg)
Empty Fields Validation	Required fields must not be empty
Invalid Input	Show error message for incorrect input

4. Orders:

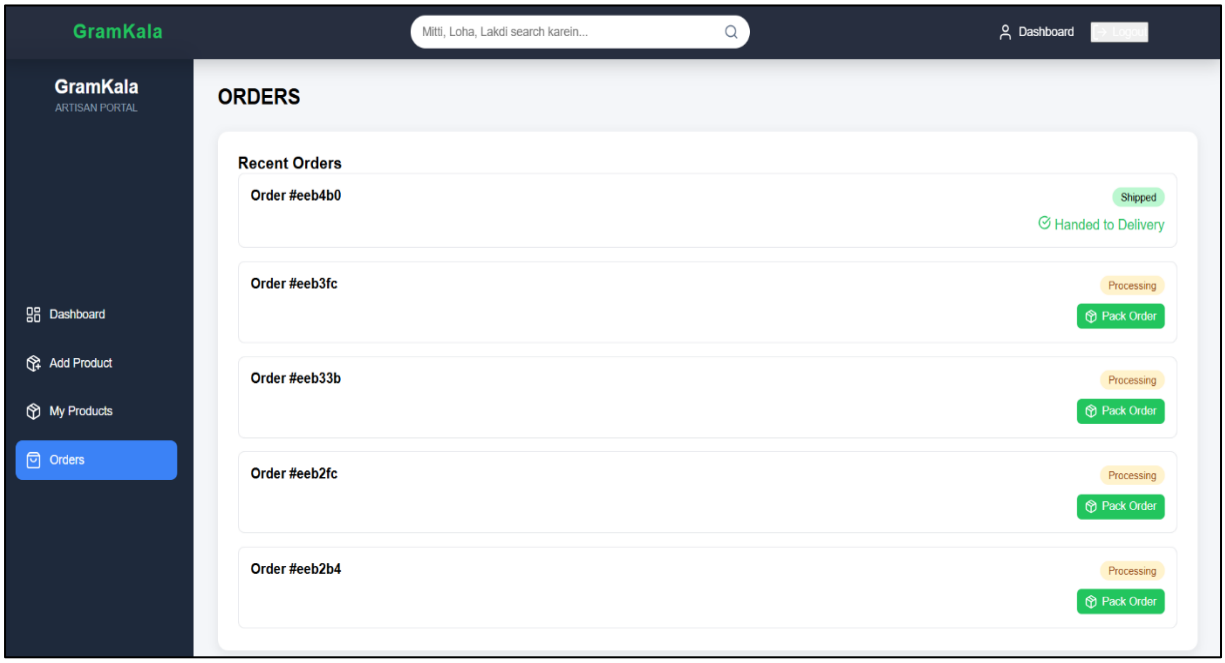

Figure 6.13 Artisan Portal Orders Page

This page displays all orders received by the artisan in a structured format for easy tracking and management. It allows the artisan to monitor order details, status, and take necessary actions on each order.

Each order shows important details such as product name, customer details, quantity, total amount, and order status. This helps the artisan to manage incoming orders efficiently.

Artisan can:

• View complete list of all received orders
• Check order details (product, customer, quantity, date, amount)
• Monitor order status (Pending, Confirmed, Delivered)
• Accept or process new orders
• Update order status (e.g., mark as delivered)
• Manage and track all orders efficiently

```
const fetchOrders = async () => {
  try {
    const res = await API.get("/orders/artisan");
    setOrders(res.data || []);
  } catch (error) {
    console.error("Error fetching orders:", error);
    alert("Failed to load orders");
  } finally {
    setLoading(false);
  }
};
```

Rural Craft Collection:

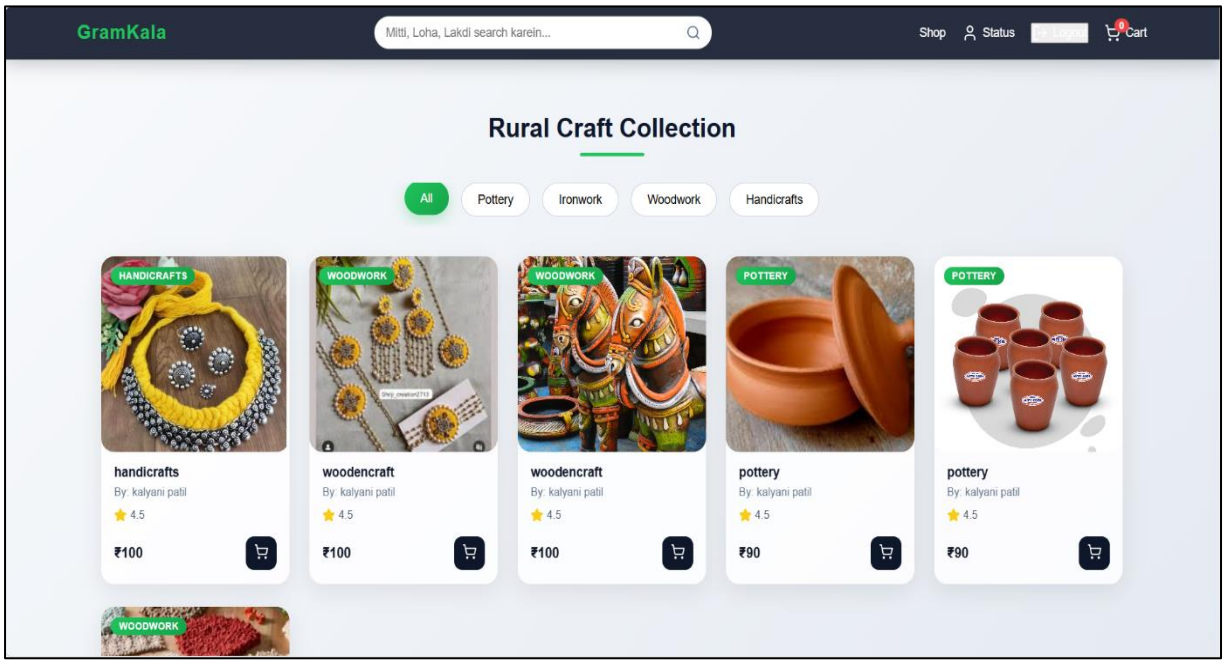

Figure 6.14 Rural Craft Collection Page

This page displays all available rural craft products for customers in a structured and attractive format. It helps users explore different handmade products created by artisans from rural areas.

Each product is displayed in a card layout showing important details such as:

• Product image

• Product name

• Category (e.g., pottery, handmade items)

• Price

• Seller (artisan) name

This page allows customers to browse products easily and select items for purchase.

Customer can:

• View all available rural craft products

• Check product details (name, category, price, artisan)

• Search products using search bar

• Add products to cart

• Explore different categories of rural crafts

```
const fetchProducts = async () => {
  try {
    const res = await API.get("/products/all");
    setProducts(res.data || []);
  } catch (error) {
    console.error("Error fetching products:", error);
    alert("Failed to load products");
  } finally {
    setLoading(false);
  }
};
```

My Orders:

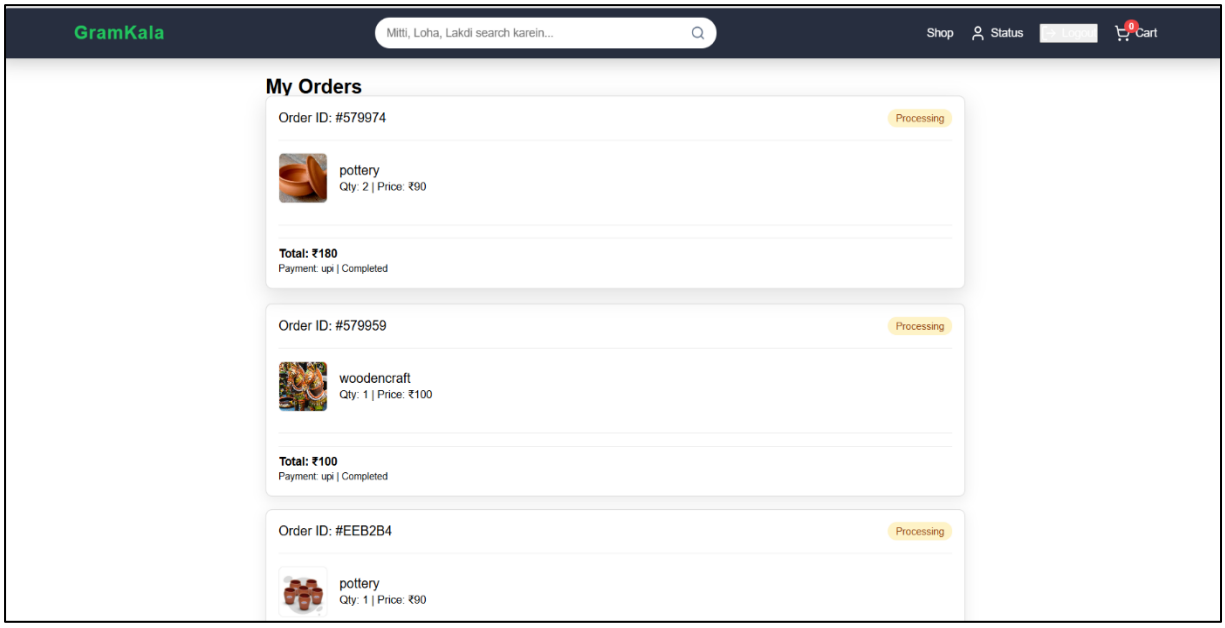

Figure 6.15 My Orders Page

This page displays all orders placed by the customer in a structured format for easy tracking and management. It helps customers view their order details, payment information, and current order status.

Each order is displayed in a card or table format showing important details such as:

• Product name

• Quantity

• Total amount

• Payment status

• Order date

• Order status

Customer can:

• View complete list of all placed orders

• Check order details (product, quantity, amount, date)

• Monitor order status (Pending, Confirmed, Delivered)

• Check payment status (Paid / Pending)

• Track current order progress

• Manage and review past orders

Status indicators are used as follows:

1. **Pending** – Order is placed but not yet confirmed
2. **Confirmed** – Order is accepted and being processed
3. **Delivered** – Order successfully completed

```
const fetchMyOrders = async () => {
  try {
    const res = await API.get("/orders/my");
    setOrders(res.data || []);
  } catch (error) {
    console.error("Error fetching orders:", error);
    alert("Failed to load orders");
  } finally {
    setLoading(false);
```

```
    }
};
```

Add to Cart Page:

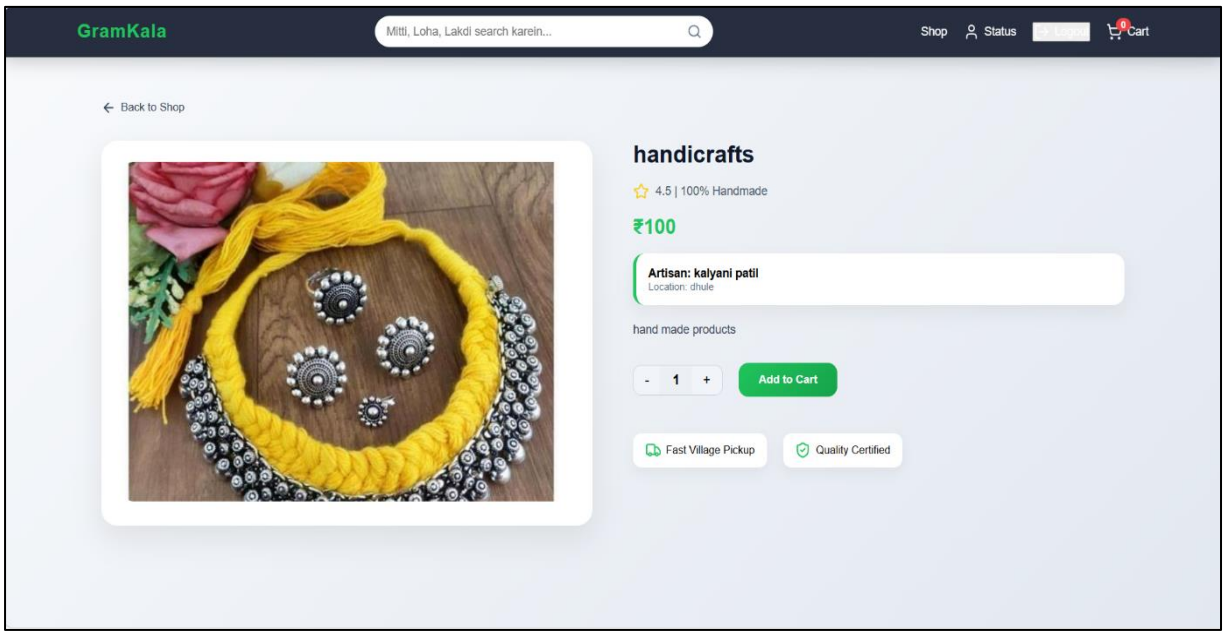

Figure 6.16 Add to Cart Page

This page allows customers to add selected products to their cart for purchase. It displays product details in a structured format and helps users manage items before placing an order.

Each product added to the cart shows important details such as:

• Product name

• Product image

• Price per item

• Quantity selected

• Total price for that item

Customer can:

• Add products to cart

• View all selected items

• Update product quantity

• Remove items from cart
• View total cart amount

```
const addToCart = async (productId) => {

  try {

    const res = await API.post("/cart/add", {

      productId,

      quantity: 1

    });

    alert("Product added to cart");

    fetchCart();

  } catch (error) {

    console.error("Error adding to cart:", error);

    alert("Failed to add product");

  }

};
```

Proceed to Checkout Page:

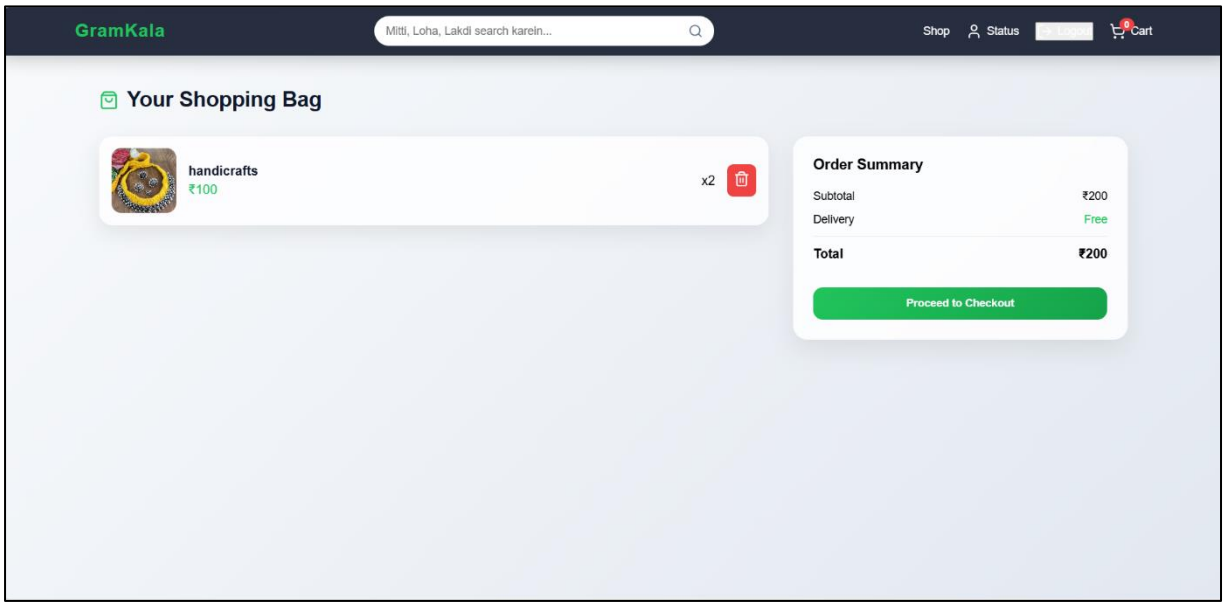

Figure 6.17 Proceed to Checkout Page

This page shows the summary of all products selected by the customer before placing the final order. It provides complete details of the cart items along with payment information.

The page includes:

• List of selected products with quantity and price

• Total amount of all items in the cart

• Customer delivery details (address, pincode, etc.)

Customer can:

• View all selected cart items

• Check total order amount

• Verify delivery address details

• Select payment method

• Proceed to place order

Payment Details:

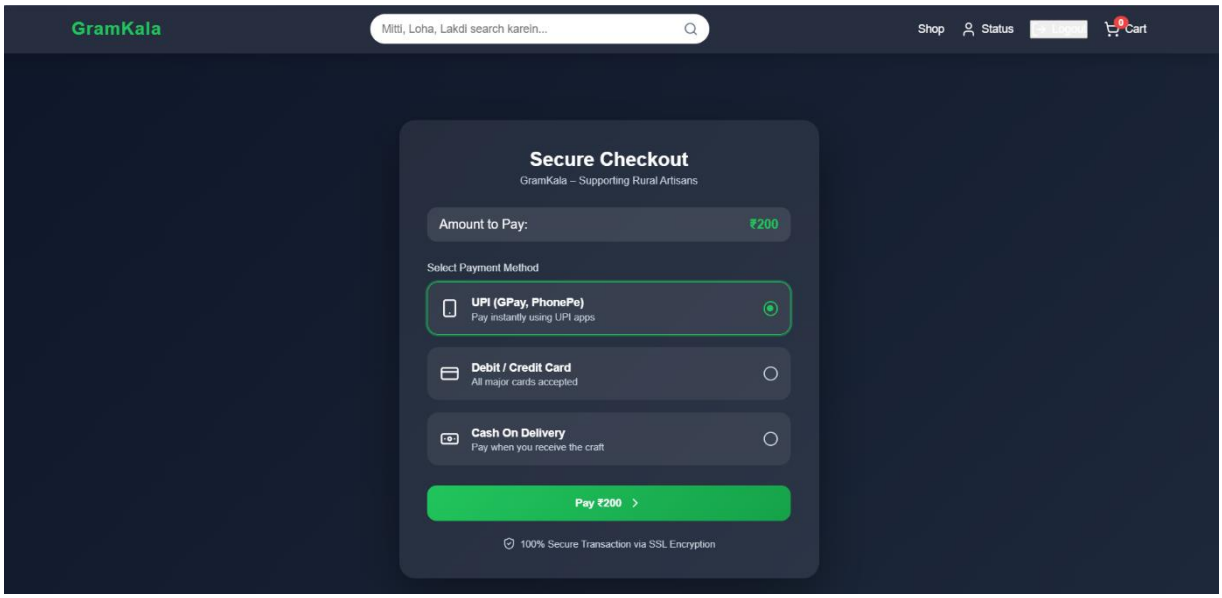

Figure 6.18 Payment Details Page

This page displays the complete payment information for the customer's order. It allows the customer to review payment details and complete the transaction securely.

The page includes:

• Total order amount

• Advance payment amount (if applicable)

• Remaining balance amount

• Selected payment method (COD / UPI)

• Payment status (Paid / Pending)

It ensures that the customer clearly understands the payment breakdown before confirming the payment.

Customer can:

• View total payable amount

• Check advance and remaining payment details

• Select or confirm payment method

• Complete payment process

• View payment status after transaction

Status indicators are used as follows:

1. **Paid** – Payment completed successfully
2. **Pending** – Payment not completed
3. **Failed** – Payment unsuccessful

```
const handlePayment = async () => {
  try {
    const res = await API.post("/payment/process", {
      orderId: order.id,
      amount: totalAmount,
      paymentMethod: selectedMethod,
    });
    alert("Payment successful!");
    navigate("/customer/my-orders");
  } catch (error) {
    console.error("Payment Error:", error.response || error);
    alert("Payment failed");
  }
};
const advance = Math.floor(totalAmount * 0.2);
const remaining = totalAmount - advance;
```

Table 7 : Payment Validations

Field Name	**Validation / Description**
Payment Method	"Required field", User must select payment method (COD / UPI)
Total Amount	"Auto-calculated field", Displayed based on cart items
Advance Amount	"Auto-calculated field", 20% of total amount
Remaining Amount	"Auto-calculated field", Total - Advance
Order ID	"Required field", Must be valid order reference
Empty Fields Validation	All required fields must not be empty
Invalid Payment Method	Show error if no payment method selected

6.2 Application of Rural E-Commerce & Logistics System:

The Rural E-Commerce & Logistics System we have developed acts as a powerful platform for connecting rural artisans with customers through an online marketplace. It simplifies the process of buying and selling handmade and local products by providing features like user registration, product management, order processing, and delivery tracking. The system helps artisans to showcase their products to a wider audience, while customers can easily browse, purchase, and track orders in real-time. Admin can efficiently manage users, products, and logistics activities, ensuring smooth system operation. Overall, this system improves accessibility, promotes rural businesses, and enhances the efficiency of the supply chain through digital technology.

CHAPTER 7

TESTING

7.1 INTRODUCTION

This chapter covers the testing activities performed for the Rural E-Commerce & Logistics System. Testing of this system is important because it serves as a platform for managing products, orders, payments, and deliveries for all users including customers, artisans, and admin. If any module fails or behaves incorrectly, the system may not function properly.

The objective of this testing phase is to verify that all functionalities work correctly across all three roles. The system should allow valid users to log in successfully, manage products, place orders, process payments, and track deliveries. It must also prevent unauthorized access and handle invalid inputs by validating all required fields. Testing was carried out for both positive scenarios (successful login, product addition, order placement, payment processing, and delivery updates) and negative scenarios (invalid login, missing data, incorrect inputs, and unauthorized access).

Testing was performed using the following tools:

• Selenium IDE: A record-and-playback tool used for quick testing of user interface scenarios.

• Selenium WebDriver: A scripting tool used for automated and repeatable test execution with proper validation.

Out of scope for this chapter:

• Advanced security testing

• Performance and load testing

• Third-party payment gateway testing

7.2 TEST PLAN

The test plan defines the objectives, scope, testing environment, tools, entry and exit criteria, and risk factors associated with testing of the Rural E-Commerce & Logistics System. This test plan is prepared to ensure a structured and systematic testing approach.

A proper test plan helps in maintaining consistency and accuracy during testing. It ensures that all modules such as User, Artisan, and Admin are tested thoroughly. The plan also helps in identifying defects early and improving overall system quality.

Testing activities were designed to validate core functionalities like user authentication, product management, order processing, payment handling, and delivery tracking. The plan ensures that the system meets the required functional and non-functional requirements effectively.

Test Plan: (Rural E-commerce & Logistics System)

Version: 1.0

Test Plan ID: TP ECOM_01

Introduction:

The test plan describes the testing process for the Rural E-commerce & Logistics System. The System allows artisan to request registration, admin to approve artisan and products, and customer to view products, add them to cart, and make payments. The system ensures and secure and correct flow from artisan approval to product purchase.

Test Items:

- Artisan registration request module
- Admin approval module
- Product upload module
- Product approval module
- Product display module
- Database storage module
- Output display module

Features to be tested:

- Artisan Registration request submission
- Admin approval/rejection to artisan
- Artisan login after approval
- Artisan upload product
- Admin approval/rejection of products
- Data Storage in database

Feature not to be Tested:

- Real Time-delivery tracking
- External API integration

- Advanced UI design
- External payment gateway integration

Approach:

- Functional and non-functional resting will be performed
- Positive and negative rest case will be executed
- Manual testing and selenium automation will be used

Pass/Fall criteria:

- All high-priority test cases must be pass successfully
- System should not have any critical or blocking defects.
- Expected results should match actual results for test.
- All major defect must be fixed

Cases:

- All major defects must be fired and retested
- If incorrect output, system crash, or errors occurs

Suspension criteria & Resumption Requirement:

Suspension criteria:

- system crash
- System failure
- Admin approval system not working
- Database connection error

Resumption:

- After fixing the issue and verifying the solution.

Test Deliverable:

- Test Plan document
- Test cases
- Defect Reports
- Test Result Reports
- Final Testing summary

Test Tasks:

- Prepare test cases
- Execute rest case

- Record results
- Identity defects
- fix bugs

Test Environment:

OS: Windows

Frontend: HTML, CSS, React

Backend: Node.js

Database: MongoDB

Responsibilities:

Developer: Develop and fix bugs.

Tester: Perform testing and report defects.

Staffing and Training:

Team member trained React.js, MongoDB Basic knowledge of testing technique required

Scheduling:

Project deadline:- 14 March 2026, 12:00PM

Risks:

- Payment failure issues
- Incorrect admin approval
- Database inconsistency

Approvals:

Project Guide, Team Member

7.3 Test Cases

Test Case ID	Objective	Input (Step)	Expected Output	Actual Output	Status
TC_001	Artisan Request	Fill Registration Form	Request send to Admin	Request Send	Pass
TC_002	Login without admin approval	Valid credentials	Login blocked With approval message	User able to Login	Fail
TC_003	Admin Approval	Approve artisan	Artisan account activate	Activated	Pass
TC_004	Artisan Login	Valid credentials	Login Successful	Login Successful	Pass
TC_005	Upload Product	Product Details	Product Submitted	Product Submitted	Pass
TC_006	Product Approval	Admin approve product	Product Visible to User	Visible	Pass
TC-007	Add to Cart	Select Product	Product Added	Added	Pass
TC-008	Reject Product	Admin Rejects	Product Not Shown	Not Shown	Pass
TC-009	View Product	Open Product Page	Approve Product Displayed	Some Product Not Display	Fail

7.4 DEFECT REPORT :-

The Artisan & Product Approval Module is an important part of the Rural E-Commerce & Logistics System that ensures only valid artisans and quality products are available on the platform. In this module, the admin reviews artisan registration requests and approves or rejects them based on verification. Similarly, products added by artisans are also reviewed by the admin before being published. This process helps in maintaining authenticity, product quality, and trust among users. Any issues in this module may affect system workflow, making it essential to test and monitor this module carefully.

Defect Report 1:-

Table 8 Defect Report (Artisan & Product Approval Module)

Defect ID:	DR001
Project Name:	Rural E-Commerce & Logistics System
Product:	Web Application
Release Version:	v1.0
Module:	Artisan & Product Approval Module
Detected Build Version:	v1.1
Summary:	Issue in artisan and product approval process.
Description:	Even after the admin approves the artisan registration and product submissions, the system does not function as expected. The approved artisans are unable to properly access their dashboard, which restricts them from managing their products and orders. Additionally, products that have been approved by the admin are not visible on the customer side, meaning they do not appear in the product listing or search results. This issue indicates a possible problem in role-based access control or data synchronization between modules.

Actual Result:	Artisan faces access issues and products are not displayed to customers
Expected Result:	Artisan should get access after approval and approved products should be visible to customers
Severity:	High (Major functionality issue affecting system workflow)
Priority:	High (Requires immediate attention and fix)
Reported By:	Test Engineer
Status:	Open

7.5 TEST PLAN

The test plan defines the objectives, scope, testing environment, tools, entry and exit criteria, and risk factors associated with testing the Artisan & Product Approval Module of the Rural E-Commerce & Logistics System. This test plan is prepared following a structured format to ensure proper testing and quality assurance.

A structured test plan ensures that testing activities are systematic, consistent, and aligned with project quality goals. It also helps in identifying defects early and improving system reliability.

Test Plan: (Rural E-commerce and logistics)

Version: 1.0

Test Plan ID: TP_APPROVAL_01

1. Introduction

This test plan verifies that the Artisan & Product Approval Module works correctly. It ensures that approved artisans can access their dashboard and approved products are visible to customers. It also checks system behavior for invalid conditions and ensures proper workflow after approval.

2. Test Items

- Artisan registration approval functionality
- Product approval functionality
- Dashboard access for approved artisans
- Product visibility on customer side
- Status update after approval
- Error handling and validation

3. Features to be Tested

- Artisan approval by admin
- Product approval by admin
- Dashboard access after approval
- Product display to customers
- Role-based access control
- Data synchronization between modules

4. Features Not to be Tested

- Login module
- Payment processing
- Order management
- Delivery tracking
- Reports and analytics

5. Approach

- Functional testing will be performed
- Positive and negative test cases will be executed
- Manual testing and Selenium automation will be used
- UI and workflow validation testing will be included
- API testing using Postman

6. Pass / Fail Criteria

- All critical and high-priority test cases must pass
- Approved artisans must access dashboard successfully
- Approved products must be visible to customers

7. Suspension Criteria and Resumption Requirements

- Testing will be suspended if critical defects are found
- Testing will resume after fixing the issues

8. Test Deliverables

- Test plan document
- Test cases document
- Defect report
- Test summary report

9. Test Tasks

- Prepare test plan and test cases
- Execute test cases
- Identify and report defects
- Retest after bug fixing
- Prepare final test report

10. Test Environment

- Operating System: Windows 11
- Browser: Google Chrome, Microsoft Edge
- Frontend: React (localhost)
- Backend: Node.js + Express.js
- Database: MongoDB
- Server: Localhost / Test Server
- Tools: Selenium IDE, Postman
- Network: Internet connection

11. Responsibilities

- QA Lead: Review artifacts
- Test Engineer: Execute tests and log defects
- Developer: Fix defects
- Project Manager: Approve release

12. Staffing and Training

Knowledge of approval workflow, testing techniques, and tools is required.

13. Scheduling

Project deadline: 20 March 2026, 12:00 PM

14. Risks

- Invalid test data
- Browser compatibility issues
- Server downtime
- Security vulnerabilities

15. Approvals

Project Guide, Team Members

Test Cases for your defect (Artisan & Product Approval Module)

Test Case ID	Objective	Input (Step)	Expected Output	Actual Output	Status
TC_001	Artisan access after approval	Register artisan Artisan login Admin approves	Artisan should access dashboard successfully	Artisan successfully accesses dashboard	Pass
TC_002	Product visibility after approval	Artisan adds product Admin approves product Customer checks product list	Approved product should be visible	Product is displayed to customer	Pass
TC_003	Artisan login validation	Approved artisan login	Login should be successful	Login successful without issues	Pass
TC_004	Product approval workflow	Add product Admin approves	Status should be “Approved” and visible	Status updated and product visible	Pass
TC_005	Role-based access control	Approve product Refresh customer page	Product should appear in real-time	Product Displayed correctly after refresh	Pass

7.6 DEFECT REPORT:-

The Product Management Module is an important part of the Rural E-Commerce & Logistics System that allows artisans to add and manage their products on the platform. In this module, artisans upload product details along with images, which are essential for displaying products to customers. Proper image upload ensures better product visibility and user engagement. However, issues in the image upload functionality may lead to products being displayed without images, affecting the overall user experience and system effectiveness. Therefore, it is important to test and monitor this module carefully to ensure smooth product listing and proper display of product images.

Defect Report 2:-

Table 9 Defect Report (Artisan & Product Management Module)

Defect ID:	DR002
Project Name:	Rural E-Commerce & Logistics System
Product:	Web Application
Release Version:	v1.0
Module:	Artisan & Product Management Module
Detected Build Version:	v1.1
Summary:	Image upload failure in product module
Description:	While adding a new product, the image upload functionality does not work as expected. When the artisan selects and uploads an image file, the system either fails to store the image in the database or does not retrieve and display it on the product listing page after submission. In some cases, the image path may not be saved correctly or the file may not be uploaded to the server directory. This results in products being displayed without images, which reduces product attractiveness and negatively impacts user experience.

Actual Result:	Image is not uploaded or not displayed with the product
Expected Result:	Image should be uploaded successfully and displayed along with the product
Severity:	Medium (Affects product display and user experience)
Priority:	High (Needs to be fixed to ensure proper product listing)
Reported By:	Test Engineer
Status:	Open

7.7 TEST PLAN

The test plan defines the objectives, scope, testing environment, tools, entry and exit criteria, and risk factors associated with testing the Artisan & Product Management Module of the Rural E-Commerce & Logistics System. This test plan is prepared following a structured format to ensure proper testing and quality assurance.

A structured test plan ensures that testing activities are systematic, consistent, and aligned with project quality goals. It also helps in identifying defects early and improving system reliability.

Test Plan: (Rural E-commerce And logistics)

Version: 1.0

Test Plan ID: TP_ PRODUCT_02

1. **Introduction**

This test plan verifies that the Artisan & Product Management Module works correctly. It ensures that artisans can add products successfully, upload images properly, and manage product details. It also checks that products are displayed correctly to customers and validates system behavior for invalid inputs.

2. **Test Items**

- Product add functionality
- Image upload functionality
- Product update and delete operations
- Product display to customers
- Data storage and retrieval
- Error handling and validation

3. Features to be Tested

- Add product with valid details
- Image upload and display
- Update product details
- Delete product
- Product visibility on user side
- File upload validation
- Data synchronization

4. Features Not to be Tested

- Login module
- Payment processing
- Order management
- Delivery tracking
- Reports and analytics

5. Approach

- Functional testing will be performed
- Positive and negative test cases will be executed
- Manual testing and Selenium automation will be used
- UI and workflow validation testing will be included
- API testing using Postman

6. Pass / Fail Criteria

- All critical and high-priority test cases must pass
- Approved artisans must access dashboard successfully
- Approved products must be visible to customers

7. Suspension Criteria and Resumption Requirements

- Testing will be suspended if critical defects are found
- Testing will resume after fixing the issues

8. Test Deliverables

- Test plan document
- Test cases document
- Defect report
- Test summary report

9. Test Tasks

- Prepare test plan and test cases
- Execute test cases
- Identify and report defects
- Retest after bug fixing
- Prepare final test report

10. Test Environment

- Operating System: Windows 11
- Browser: Google Chrome, Microsoft Edge
- Frontend: React (localhost)
- Backend: Node.js + Express.js
- Database: MongoDB
- Server: Localhost / Test Server
- Tools: Selenium IDE, Postman
- Network: Internet connection

11. Responsibilities

- QA Lead: Review artifacts
- Test Engineer: Execute tests and log defects
- Developer: Fix defects
- Project Manager: Approve release

12. Staffing and Training

Knowledge of product management workflow, file upload handling, and testing tools is required.

13. Scheduling

Project deadline: 20 March 2026, 12:00 PM

14. Risks

- File upload failure issues
- Data storage errors
- Browser compatibility issues
- Server downtime

15. Approvals

Project Guide, Team Members

Test Cases for your defect (Artisan & Product Management Module)

Test Case ID	Objective	Input (Step)	Expected Output	Actual Output	Status
TC_001	Add product with valid details	Login as artisan Go to Add Product Enter valid details Submit	Product should be added successfully	Product added successfully	Pass
TC_002	Upload valid product image	Add product Upload valid image file Submit	Image should upload and display correctly	Image displayed correctly	Pass
TC_003	Upload invalid file format	Upload non-image file	System should show error message	Error message displayed	Pass
TC_004	Image upload failure (defect case)	Upload image Submit product	Image should be visible	Image not displayed	Fail
TC_005	Update product details	Select product Edit details	Product details should update	Product updated successfully	Pass

7.8 DEFECT REPORT:-

The Payment Module is a crucial part of the Rural E-Commerce & Logistics System that allows customers to complete transactions for purchasing products. It ensures secure and successful transfer of payment from the customer to the platform. A properly functioning payment system is essential for order confirmation, revenue generation, and user trust. However, issues in the payment process may lead to transaction failures, incomplete orders, or loss of customer confidence. Therefore, it is important to test and monitor this module carefully to ensure smooth and secure payment processing.

Defect Report 3:-

Table 10 Defect Report (Payment process failure)

Defect ID:	DR003
Project Name:	Rural E-Commerce & Logistics System
Product:	Web Application
Release Version:	v1.0
Module:	Payment process failure
Detected Build Version:	v1.1
Summary:	Payment process failure
Description:	The payment functionality of the system does not work as expected. After adding products to the cart and proceeding to the payment page, the customer is unable to complete the transaction. The payment page either shows an error or fails to load properly. This issue may be caused due to incorrect API integration, server-side errors, or payment gateway issues. As a result, orders cannot be placed successfully, which directly affects business operations and user experience.

Actual Result:	Payment page shows error or does not load
Expected Result:	Payment should be successful and order should be placed
Severity:	High (Critical functionality failure)
Priority:	High (Needs immediate fix)
Reported By:	Test Engineer
Status:	Open

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion:

The Rural E-Commerce & Logistics System successfully provides a platform to connect rural artisans with customers through an efficient online marketplace. The system simplifies product management, order processing, payment handling, and logistics tracking in a structured manner. It reduces the gap between rural sellers and urban buyers by providing better visibility and accessibility to handmade and local products. With features like user management, product listing, order tracking, and delivery management, the system ensures transparency, efficiency, and ease of use for all stakeholders. Overall, the project achieves its objective of promoting rural businesses and improving the supply chain through digital technology.

8.2 Future Scope:

The Rural E-Commerce & Logistics System can be further enhanced with advanced features to improve usability and efficiency. In the future, the system can include mobile application support for better accessibility among rural users. Integration of online payment gateways with more secure and faster transactions can be implemented. Real-time GPS tracking for delivery can be added to improve logistics transparency. The system can also include AI-based product recommendations and demand forecasting to help artisans increase sales. Multi-language support can be introduced to make the platform more user-friendly for rural communities. Additionally, expanding the system to support a larger number of users and integrating government schemes or support programs can further promote rural businesses and strengthen the overall ecosystem.

CHAPTER 9

REFERENCE AND BIBLIOGRAPHY

1. Reference:

- **Books**

  1. Software Engineering: A Practitioner's Approach (Eighth Edition by Roger S. Pressman & Bruce R. Maxim)

- **Websites**

  1. HTML, CSS & JavaScript Tutorial – www.w3schools.com
  2. React.js Documentation – https://reactjs.org
  3. Node.js Documentation – https://nodejs.org
  4. MongoDB Documentation – https://www.mongodb.com
  5. Express.js Guide – https://expressjs.com

2. Bibliography:

Rural E-Commerce And Logistics System:

Study of Rural E-Commerce Development and Logistics Management

DEPARTMENT OF COMPUTER ENGINEERING

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule.

Websites:

https://www.mongodb.com: MongoDB is used in the project for managing and storing application data efficiently. It helps in maintaining records of users, products, and transactions, and supports easy data retrieval and updates

https://reactjs.org: React is used in the project for designing and developing the frontend user interface of the application. It helps in creating interactive, dynamic, and reusable components, providing a smooth and user-friendly experience.

https://nodejs.org: Node.js is used in the project for building the backend server and handling application logic. It manages client requests, processes data, and communicates with the database.

https://developer.mozilla.org: MDN Web Docs is used as a reference for learning HTML, CSS, and JavaScript concepts. It provides detailed documentation, examples, and best practices that help in understanding and implementing web development features effectively.